1. PART 1 Embedded systems definitions and main characteristics. Embedded systems control unit implementation options and characteristics (integration options, typically peripherals, programming techniques). Implementing the interaction between the system and the user. PART 2 Program units. Subprograms. Parameter evaluation. Parameter passing methods. Block. Scoping, accessibility. Abstract data type. Generic programming. I/O tools of programming languages, file handling. Exception handling. Parallel programming.

Part 1

Embedded Systems Definition

An embedded system is a computer system that has a dedicated function within a larger mechanical or electronic system. Unlike general-purpose computers, embedded systems are designed to perform specific tasks with real-time constraints. They combine hardware and software to form a complete system.

Main characteristics of embedded systems:

1. **Application-specific** – Designed for a particular task or set of tasks

2. **Real-time operation** – Must respond to events within strict time constraints
3. **Resource constraints** – Limited memory, processing power, and energy
4. **Reliability** – Must operate continuously without failure
5. **Reactive systems** – Respond to external events from the environment
6. **Cross-platform development** – Software is developed on a host system and deployed to the target

Control Unit Implementation Options

The control unit in embedded systems can be implemented using several approaches:

1. **Microprocessor-based systems** – Uses a general-purpose microprocessor with external memory and peripherals. Offers flexibility but requires more components.

2. **Microcontroller-based systems** – Integrates processor, memory, and I/O on a single chip. More compact and power-efficient, ideal for embedded applications.

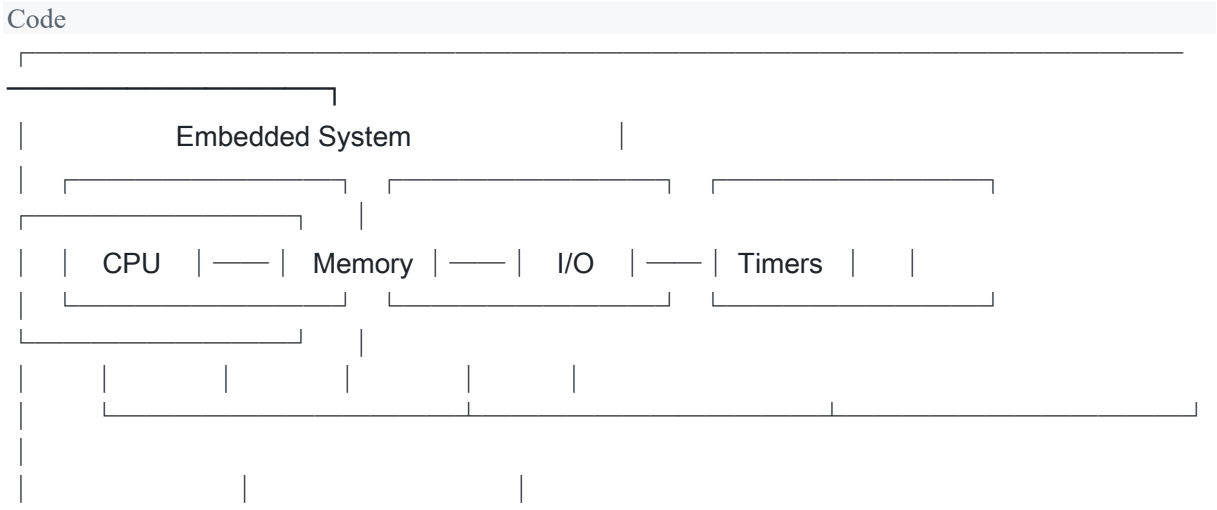
3. **FPGA-based systems** – Uses programmable logic for hardware-level customization. Offers parallelism and can be reconfigured.

4. **ASIC-based systems** – Application-specific integrated circuits designed for a single purpose. Highest performance but expensive to develop.

5. **DSP-based systems** – Digital Signal Processors optimized for signal processing applications like audio and video.

Integration Options

Code



Typical Peripherals

1. **Serial Communication Interfaces (SCI)** – UART, USART for asynchronous serial communication
2. **Inter-Integrated Circuit (I2C)** – Two-wire interface for short-distance communication
3. **Serial Peripheral Interface (SPI)** – High-speed synchronous serial communication
4. **Universal Serial Bus (USB)** – Standard interface for data transfer and power supply
5. **GPIO (General Purpose Input/Output)** – Digital pins for interfacing with external devices
6. **ADC/DAC** – Analog-to-Digital and Digital-to-Analog converters
7. **PWM (Pulse Width Modulation)** – For motor control and dimming applications
8. **Timers/Counters** – For timing operations and event counting

Programming Techniques

1. **Polling** – Continuously checking the status of peripherals
2. **Interrupt-driven** – Responding to hardware interrupts for efficient processing
3. **DMA (Direct Memory Access)** – Hardware-based data transfer without CPU intervention
4. **Real-Time Operating Systems (RTOS)** – For managing tasks with timing constraints

User Interaction Implementation

User interaction in embedded systems is achieved through:

1. **Input devices** – Buttons, keyboards, touchscreens, sensors
2. **Output devices** – LEDs, LCDs, displays, speakers, actuators
3. **Human-Machine Interface (HMI)** – Graphical interfaces for complex systems
4. **Communication interfaces** – Bluetooth, Wi-Fi, cellular for remote interaction

---

Part 2

Program Unit

A program unit is a sequence of one or more lines organized as statements, comments, and directives. Program units can be:

- **Main program** – The entry point of execution
- **Module** – A collection of related procedures and data
- **External procedure** – A separately compiled subprogram
- **Block data** – A unit for initializing data in named common blocks

Subprogram

A subprogram is a sequence of instructions invoked from remote locations in a program. When the subprogram execution completes, control returns to the calling location.

Types of subprograms:

1. **Functions** – Return a value and can be used in expressions
2. **Procedures/Subroutines** – Perform actions but do not return a value

Parameter Evaluation

Parameter evaluation maps formal and actual parameters when a subprogram is called:

1. **Order-based assignment** – Parameters are assigned based on their position in the list
2. **Name-based assignment** – Parameters are explicitly named (keyword arguments)
3. **Type checking** – Some languages require exact type matching; others allow implicit conversion

Parameter Passing Methods

1. **Pass by value** – A copy of the actual parameter is passed. Changes to the formal parameter do not affect the original.

```
C
void increment(int x) {
```

```
    x = x + 1;  // Original unchanged
}
```

2. **Pass by reference** – A reference to the original variable is passed. Changes affect the original.

```
C
void increment(int *x) {
    *x = *x + 1;  // Original is modified
}
```

3. **Pass by value-result** – Value is copied in, modified, and copied back at the end
4. **Pass by name** – The actual parameter expression is substituted directly (used in macro expansion)

Block

A block is a structured group of source code containing declarations and statements. Blocks define scope boundaries and allow local variable declarations.

```
C
{
    int localVar = 10;  // Block-local variable
    // statements
}  // localVar goes out of scope
```

Scoping and Accessibility

Scope defines where an identifier (variable, function, etc.) can be accessed:

1. **Global scope** – Accessible throughout the entire program
2. **Local scope** – Accessible only within the defining block
3. **Static scope (Lexical)** – Determined at compile time based on program structure
4. **Dynamic scope** – Determined at runtime based on calling sequence

Accessibility levels in OOP:

- **Public** – Accessible from anywhere
- **Private** – Accessible only within the class

- **Protected** – Accessible within the class and subclasses

Abstract Data Type (ADT)

An ADT defines a data type by its behavior (operations) rather than its implementation. It specifies:

1. **Data** – The values that can be stored
2. **Operations** – The functions that can be performed
3. **Encapsulation** – Implementation details are hidden

Examples: Stack (push, pop, isEmpty), Queue (enqueue, dequeue), List (insert, delete, find)

Generic Programming

Generic programming writes algorithms in terms of types to work with any data type:

```Java
public class Box<T> {
    private T content;
    public void set(T content) { this.content = content; }
    public T get() { return content; }
}
```

Benefits:

- Code reusability
- Type safety at compile time
- Reduced code duplication

I/O Tools and File Handling

Programming languages provide I/O operations for:

1. **Console I/O** – Reading from and writing to standard input/output
2. **File I/O** – Reading from and writing to files

```Python
# File handling example
with open('file.txt', 'r') as f:
    content = f.read()  # Reading
```

```
with open('output.txt', 'w') as f:
    f.write('Hello')    # Writing
```

File operations: Open, Read, Write, Seek, Close

Exception Handling

Exception handling manages runtime errors without crashing the program:

```
Java
try {
    int result = 10 / 0;  // May throw exception
} catch (ArithmeticException e) {
    System.out.println("Division by zero!");
} finally {
    // Always executes
}
```

Components:

- **try** – Code that might throw an exception
- **catch** – Handler for specific exception types
- **finally** – Cleanup code that always executes
- **throw** – Explicitly raise an exception

Parallel Programming

Parallel programming executes multiple operations simultaneously using multiple processors or threads:

Paradigms:

1. **Shared memory** – Threads share a common address space (OpenMP, Pthreads)
2. **Message passing** – Processes communicate via messages (MPI)
3. **Data parallelism** – Same operation on different data elements (SIMD)

Challenges:

- Race conditions
- Deadlocks

- Synchronization overhead
- Load balancing

---

2. PART 1 Synthesis of continuous time control systems. The gain and phase margin. Linear systems and their description in time- and frequency domains. Signal transfer in control systems. PART 2 Describe the protocol data units (PDU) of the TCP and UDP transport layer protocols and the characteristics and differences of their mechanisms.

Part 1

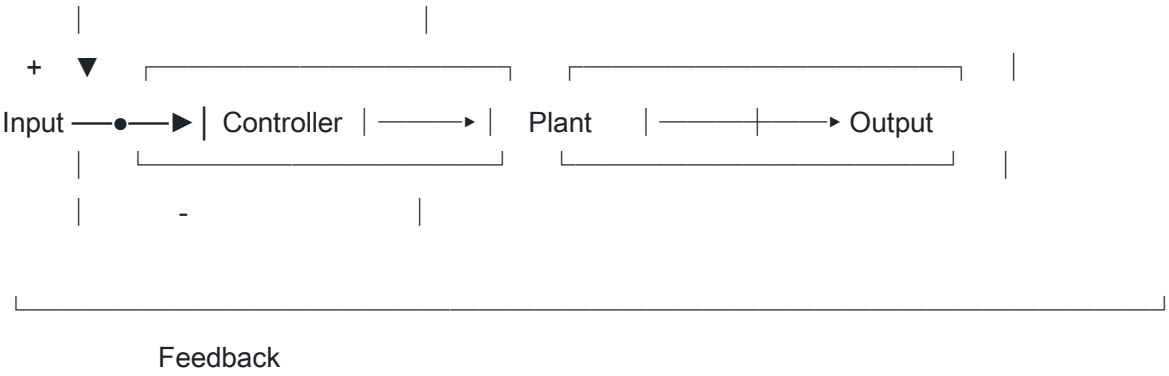
Control Systems

A control system is a system designed to regulate the behavior of other systems. It uses control loops to maintain a desired output despite disturbances.

Types of control systems:

1. **Open-loop control** – Output does not affect the control action
2. **Closed-loop (feedback) control** – Output is measured and compared to the setpoint

Code



Synthesis of Continuous Time Control Systems

The synthesis process involves:

1. **System modeling** – Creating a mathematical model (transfer function, state-space)
2. **Controller design** – Choosing controller type (P, PI, PID) and tuning parameters
3. **Stability analysis** – Ensuring the system is stable (Routh-Hurwitz, Nyquist)
4. **Performance optimization** – Meeting specifications for settling time, overshoot, steady-state error

Gain and Phase Margin

These margins indicate how close a system is to instability:

Gain Margin (GM):

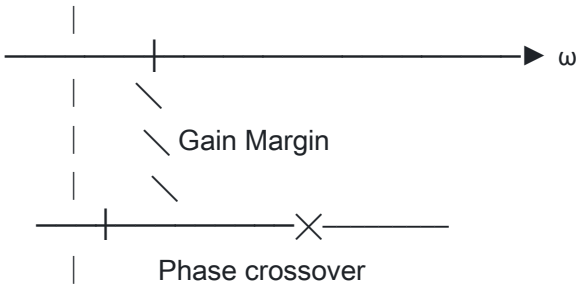
- The factor by which the gain can be increased before the system becomes unstable
- Measured at the phase crossover frequency (where phase = -180°)
- GM = 1/|G(jω)| at phase crossover
- Typically expressed in decibels (dB)

Phase Margin (PM):

- The additional phase lag that would make the system unstable
- Measured at the gain crossover frequency (where |G(jω)| = 1)
- PM = 180° + ∠G(jωc)

Code

```
Gain (dB)
   ▲
```



Design guidelines:

- GM > 6 dB (factor of 2)
- PM > 30° to 60°

Linear Systems

A linear system satisfies the principles of:

1. **Superposition** – Response to sum of inputs equals sum of individual responses
2. **Homogeneity** – Scaling input scales output by the same factor

Time-domain description:

- Differential equations
- Impulse response h(t)
- Convolution: y(t) = x(t) * h(t)

Frequency-domain description:

- Transfer function H(s) = Y(s)/X(s)
- Frequency response H(jω)
- Bode plots (magnitude and phase vs. frequency)

Signal Transfer in Control Systems

The transfer function describes how input signals are transformed to outputs:

Transfer Function: G(s) = Y(s)/X(s)

For a typical second-order system:

```
Code
```

$$G(s) = \omega_n^2 / (s^2 + 2\zeta\omega_n \cdot s + \omega_n^2)$$

Where:

- ωn = natural frequency
- ζ = damping ratio

Time-domain characteristics:

- Rise time, settling time, overshoot, steady-state error

Part 2

TCP (Transmission Control Protocol)

TCP is a connection-oriented, reliable transport layer protocol. It ensures data integrity and ordered delivery.

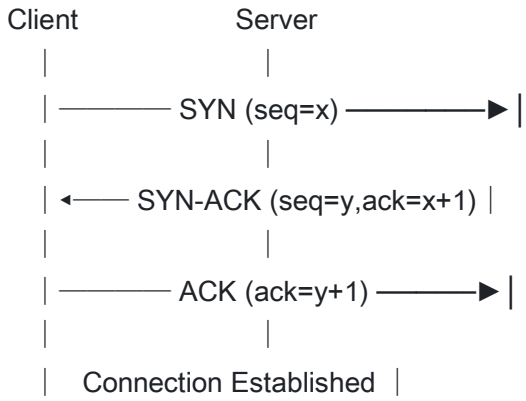
TCP Header Structure (20-60 bytes):

Field	Size	Description
Source Port	16 bits	Sender's port number
Destination Port	16 bits	Receiver's port number
Sequence Number	32 bits	Position of first byte in segment
Acknowledgment Number	32 bits	Next expected byte number
Data Offset	4 bits	Header length (in 32-bit words)
Reserved	3 bits	Reserved for future use
Flags	9 bits	Control flags (SYN, ACK, FIN, RST, etc.)
Window Size	16 bits	Flow control - receive buffer size
Checksum	16 bits	Error detection

Field	Size	Description
Urgent Pointer	16 bits	Points to urgent data
Options	Variable	Additional features (MSS, timestamps, etc.)

TCP Three-Way Handshake:

Code



TCP Mechanisms:

- Reliable delivery with acknowledgments and retransmissions
- Flow control with sliding window
- Congestion control (slow start, congestion avoidance)
- Ordered delivery using sequence numbers

UDP (User Datagram Protocol)

UDP is a connectionless, unreliable transport protocol optimized for speed and low overhead.

UDP Header Structure (8 bytes):

Field	Size	Description
Source Port	16 bits	Sender's port number
Destination Port	16 bits	Receiver's port number

Field	Size	Description
Length	16 bits	Total datagram length (header + data)
Checksum	16 bits	Error detection (optional in IPv4)

Differences Between TCP and UDP

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Guaranteed delivery	Best-effort delivery
Ordering	Ordered delivery	No ordering guarantee
Header size	20-60 bytes	8 bytes
Speed	Slower	Faster
Flow control	Yes (sliding window)	No
Congestion control	Yes	No
Use cases	Web, email, file transfer	Streaming, gaming, DNS, VoIP
Broadcasting	No	Yes
State	Stateful	Stateless

---

3. PART 1 Combinational logic design. Multiplexers/Demultiplexers.

Encoders/Decoders. Comparators. Parity generators/checkers. Arithmetical logical units. PART 2 Problem solving with search, uninformed and heuristic search algorithms. Constraint satisfaction problems.

Part 1

Combinational Logic

Combinational logic circuits produce outputs that depend only on the current inputs. There is no memory or feedback.

Design representations:

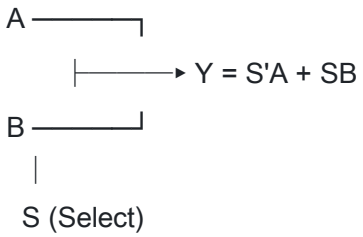
1. **Boolean Algebra** – Algebraic expressions using AND, OR, NOT operations
2. **Truth Table** – Lists all input combinations and corresponding outputs
3. **Logic Diagram** – Graphical representation using gate symbols

Multiplexers (MUX)

A multiplexer selects one of several input signals and forwards it to a single output line.

2: 1 Multiplexer:

Code



Formula: For an n: 1 MUX with n inputs, $\log_2(n)$ select lines are needed.

Truth Table for 2:1 MUX:

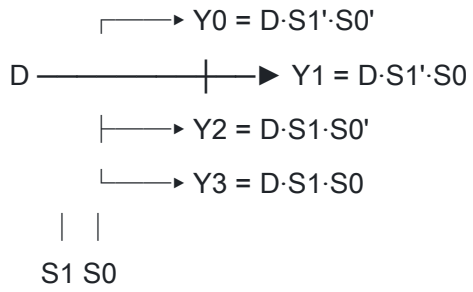
S	Y
0	A
1	B

Demultiplexers (DEMUX)

A demultiplexer takes a single input and routes it to one of several outputs based on select lines.

1:4 Demultiplexer:

Code



Encoders

Encoders convert 2^n input lines into n output lines. Only one input should be active at a time.

8:3 Priority Encoder:

Input	Output
D7=1	111
D6=1	110
D5=1	101
...	...

Input	Output
D0=1	000

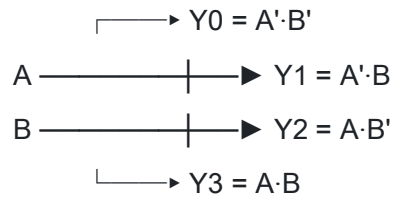
Priority encoders handle multiple active inputs by selecting the highest priority.

Decoders

Decoders convert n input lines into 2^n output lines. Exactly one output is active for each input combination.

2:4 Decoder:

Code



Digital Comparators

Comparators compare two binary numbers and indicate their relationship (equal, greater, less).

1-bit Comparator outputs:

- A = B: Y = A XNOR B
- A > B: Y = A·B'
- A < B: Y = A'·B

Multi-bit comparators chain single-bit comparators using cascading inputs.

Parity Generators/Checkers

Parity bits detect single-bit errors in data transmission.

Types:

- **Even parity** – Total number of 1s (including parity bit) is even
- **Odd parity** – Total number of 1s is odd

Parity Generator: XOR of all data bits

Code

```
P = D0 ⊕ D1 ⊕ D2 ⊕ D3 (for even parity)
```

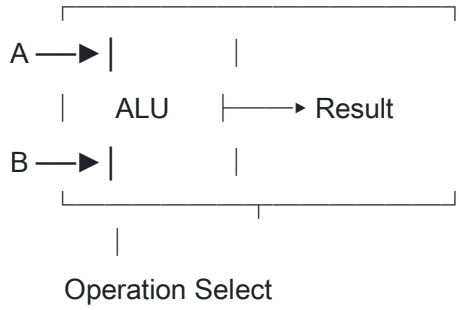
Arithmetic Logic Unit (ALU)

The ALU performs arithmetic and bitwise operations on binary numbers.

Operations:

- Arithmetic: ADD, SUB, INC, DEC
- Logical: AND, OR, XOR, NOT
- Comparison: Equal, Less than, Greater than
- Shift: Left shift, Right shift

Code



Part 2

Problem Solving with Search

In computer science, many problems can be formulated as searching through a state space to find a solution.

Components of a search problem:

1. **Initial state** – Starting point
2. **Actions** – Possible operations from each state
3. **Transition model** – Result of applying an action
4. **Goal test** – Determines if current state is a goal
5. **Path cost** – Cost of the path to the current state

Uninformed Search Algorithms

Uninformed (blind) search algorithms have no additional information about states beyond the problem definition.

1. Breadth-First Search (BFS):

- Explores all nodes at current depth before moving deeper
- **Complete:** Yes (if branching factor is finite)
- **Optimal:** Yes (for uniform cost)
- **Time complexity:** O(b^d)
- **Space complexity:** O(b^d)

2. Depth-First Search (DFS):

- Explores deepest node first, backtracks when necessary
- **Complete:** No (can get stuck in infinite paths)
- **Optimal:** No
- **Time complexity:** O(b^m)
- **Space complexity:** O(bm)

3. Uniform Cost Search:

- Expands the node with lowest path cost
- **Complete:** Yes
- **Optimal:** Yes

4. Iterative Deepening:

- Combines DFS space efficiency with BFS completeness
- Performs DFS with increasing depth limits

Heuristic (Informed) Search Algorithms

Heuristic search uses domain knowledge to guide the search.

Heuristic function h(n): Estimates cost from node n to the goal.

1. Greedy Best-First Search:

- Expands node that appears closest to goal
- f(n) = h(n)
- Not optimal, not always complete

*2. A Search:**

- Combines path cost and heuristic
- f(n) = g(n) + h(n)
- **Optimal** if h(n) is admissible (never overestimates)
- **Complete** if h(n) is consistent

Admissible heuristic: h(n) ≤ h*(n) where h*(n) is true cost

Constraint Satisfaction Problems (CSP)

CSPs define problems in terms of variables, domains, and constraints.

Components:

1. **Variables (X):** $\{X_1, X_2, ..., X_n\}$
2. **Domains (D):** Possible values for each variable
3. **Constraints (C):** Relations that variables must satisfy

Example: Map Coloring

- Variables: Regions {A, B, C, D}
- Domains: {Red, Green, Blue}
- Constraints: Adjacent regions must have different colors

CSP Solving Techniques:

1. **Backtracking Search** – Assign values and backtrack on constraint violation
2. **Constraint Propagation** – Reduce domains based on constraints
3. **Arc Consistency (AC-3)** – Ensure all arcs are consistent
4. **Heuristics:**
    - Minimum Remaining Values (MRV)
    - Degree heuristic
    - Least Constraining Value

---

4. PART 1 The SSH protocol, key generation, SSH configuration settings. PART 2 The principles of control,

feedback control and open loop control. Set point control and reference signal tracking, the role of negative feedback. Requirements for control systems.

Part 1

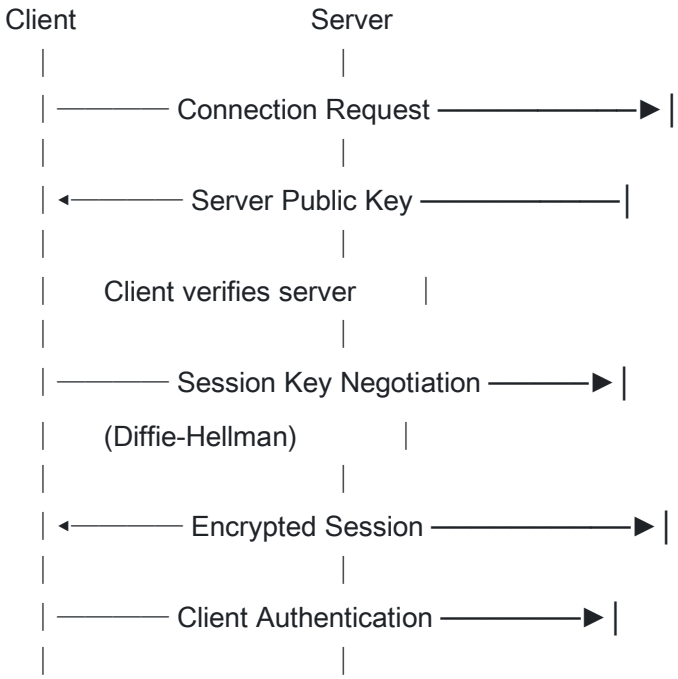
SSH Protocol

SSH (Secure Shell) is a cryptographic network protocol for secure communication over unsecured networks. It provides:

1. **Strong authentication** – Password and public key authentication
2. **Data integrity** – Ensures data is not modified in transit
3. **Encryption** – All communication is encrypted
4. **Secure tunneling** – Forward ports securely

SSH Connection Process:

Code



Steps:

1. **Server verification** – Client checks server's public key against known hosts
2. **Session key generation** – Diffie-Hellman algorithm creates shared symmetric key
3. **Client authentication** – Password or public key authentication

Key Generation

SSH uses asymmetric cryptography with public-private key pairs.

Generating SSH Keys:

```bash
ssh-keygen -t rsa -b 4096 -C "email@example.com"
```

Options:

- `-t` – Key type (rsa, ed25519, ecdsa)
- `-b` – Key size in bits
- `-C` – Comment (usually email)
- `-f` – Output file path

Key files:

- `~/.ssh/id_rsa` – Private key (keep secret!)
- `~/.ssh/id_rsa.pub` – Public key (share with servers)
- `~/.ssh/authorized_keys` – Server file listing allowed public keys
- `~/.ssh/known_hosts` – Client file with known server fingerprints

SSH Configuration Settings

Client configuration (`~/.ssh/config`):

```
Code
Host myserver
    HostName server.example. com
    User admin
    Port 2222
    IdentityFile ~/.ssh/id_rsa_custom
    ForwardAgent yes
```

Server configuration (/etc/ssh/sshd_config):

```
Code
```

```
Port 22
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
MaxAuthTries 3
AllowUsers admin developer
```

Common settings:

- Port – SSH port number (default: 22)
- PermitRootLogin – Allow/deny root login
- PasswordAuthentication – Enable password login
- PubkeyAuthentication – Enable key-based authentication
- X11Forwarding – Enable X11 graphical forwarding

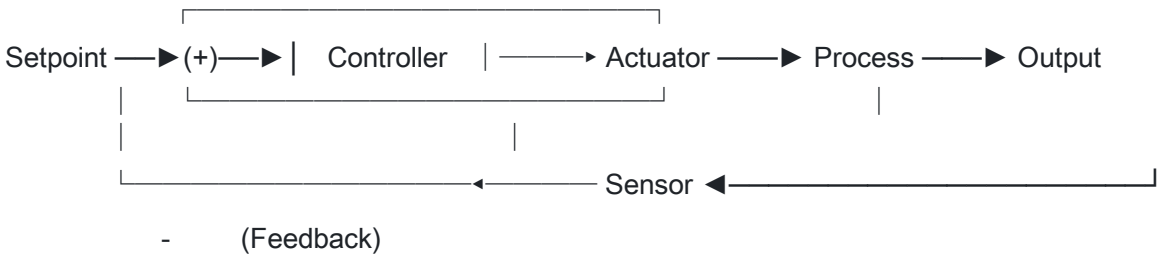
Part 2

Principles of Control

Control systems manage, command, direct, or regulate the behavior of other systems. The fundamental goal is to make a system behave in a desired manner.

Control Loop Components:

```
Code
```



1. **Sensor** – Measures the process variable (PV)
2. **Controller** – Compares PV to setpoint, calculates control action
3. **Actuator** – Implements the control action
4. **Process** – The system being controlled

Open Loop Control

In open-loop control, the controller does not receive feedback about the output.

Characteristics:

- Simple and inexpensive
- No error correction
- Sensitive to disturbances
- Calibration-dependent

Example: A toaster with a timer – it runs for a set time regardless of how done the toast is.

```
Code
Input ——► Controller ——► Process ——► Output
```

Closed Loop (Feedback) Control

Closed-loop control uses feedback to compare output with the desired setpoint.

Characteristics:

- More accurate
- Compensates for disturbances
- More complex
- Can become unstable if poorly designed

Example: A thermostat that measures temperature and adjusts heating accordingly.

Set Point Control and Reference Signal Tracking

Setpoint control:

- Maintains a constant output value
- The reference is fixed
- Example: Maintaining room temperature at 22°C

Reference signal tracking:

- Output follows a changing reference signal
- The reference varies over time
- Example: A robot arm following a trajectory

The Role of Negative Feedback

Negative feedback subtracts the output from the input to create an error signal.

Benefits:

1. **Stability** – Reduces oscillations and maintains equilibrium
2. **Reduced sensitivity** – Less affected by parameter variations
3. **Disturbance rejection** – Compensates for external disturbances
4. **Improved accuracy** – Reduces steady-state error

Negative feedback formula:

```
Code
Error = Setpoint - Measured Output
Control Action = f(Error)
```

Requirements for Good Control Systems (BASSONS)

1. **Bandwidth** – Wide range of operating frequencies; higher bandwidth means faster response
2. **Accuracy** – Low tolerance for errors; the output should closely match the setpoint
3. **Sensitivity** – The system should be sensitive to input changes but insensitive to parameter variations and noise
4. **Stability** – The system must return to equilibrium after disturbances; output should not oscillate indefinitely
5. **Oscillation** – Minimal fluctuations in output; excessive oscillation reduces stability
6. **Noise** – High tolerance to noise; external disturbances should not significantly affect output
7. **Speed** – Fast response to input changes; quick settling time

---

5. PART 1 Two-person games and their representations. Winning strategy, optimal decisions in games. PART 2 MOS transistor: large signal model and characteristics. The MOS transistor as a switch. CMOS inverter, basic logic gates. The operational amplifier. Negative feedback. Basic applications.

Part 1

Two-Person Games

Two-person games are competitive scenarios where two agents (players) take turns making decisions, each trying to maximize their own outcome.

Game Representations:

1. **Game Tree:**
   - Nodes represent game states
   - Edges represent moves/actions
   - Leaves represent terminal states with utility values

```
Code
```

```
        (MAX)
        /   \
     (MIN)  (MIN)
     / \    / \
    3   5  2   9
```

2. **Extensive Form:**
   - Shows sequence of moves
   - Indicates information available at each decision point

Types of Games:

Type	Description	Example
Perfect Information	Players see entire game state	Chess, Checkers
Imperfect Information	Hidden information exists	Poker, Battleship
Deterministic	No randomness	Chess, Tic-tac-toe
Non-deterministic	Includes chance elements	Backgammon, Monopoly

Zero-Sum Games

In zero-sum games, one player's gain equals the other's loss. The sum of utilities is always zero.

Components:

1. **Initial state** – Starting configuration
2. **Players** – Who moves at each state
3. **Actions** – Legal moves from each state
4. **Result function** – Transition model
5. **Terminal test** – Determines if game is over
6. **Utility function** – Numerical value for terminal states

Winning Strategy and Optimal Decisions

Minimax Algorithm:

The minimax algorithm finds optimal play for deterministic, perfect-information games.

- **MAX player** – Tries to maximize utility
- **MIN player** – Tries to minimize utility

Code

```
function minimax(node, depth, isMaximizing):
    if terminal(node):
        return utility(node)

    if isMaximizing:
        value = -∞
        for each child of node:
            value = max(value, minimax(child, depth-1, false))
        return value
    else:
        value = +∞
        for each child of node:
            value = min(value, minimax(child, depth-1, true))
        return value
```

Alpha-Beta Pruning:

Optimization that eliminates branches that cannot affect the final decision.

- **α (alpha)** – Best value MAX can guarantee (lower bound)
- **β (beta)** – Best value MIN can guarantee (upper bound)
- **Pruning condition:** If α ≥ β, stop exploring that branch

Benefits:

- Can reduce time complexity from O(b^d) to O(b^(d/2)) in best case
- Allows deeper search in the same time

Part 2

MOS Transistor

MOSFET (Metal-Oxide-Semiconductor Field-Effect Transistor) is a voltage-controlled device with four terminals: Gate (G), Source (S), Drain (D), and Substrate/Body (B).

Types:

1. **NMOS (n-channel):**

- Conducts when gate voltage is HIGH
- Good at passing logic 0

2. **PMOS (p-channel):**
    - Conducts when gate voltage is LOW
    - Good at passing logic 1

Modes of Operation:

Mode	Condition	Behavior
Cutoff	VGS < VTH	No conduction (OFF)
Linear (Triode)	VGS > VTH, VDS < VGS - VTH	Acts like resistor
Saturation	VGS > VTH, VDS > VGS - VTH	Current source

Large Signal Model

The large-signal model describes transistor behavior for large voltage swings where nonlinear effects are significant.

Drain current in saturation (NMOS):

Code

$$ID = (1/2) * \mu n * Cox * (W/L) * (VGS - VTH)^2$$

Where:

- μn = electron mobility
- Cox = gate oxide capacitance
- W/L = width-to-length ratio
- VTH = threshold voltage

MOS Transistor as a Switch

MOSFETs make excellent switches for digital circuits:

NMOS switch:

- Gate HIGH → Switch ON (conducts)

- Gate LOW → Switch OFF (open)

PMOS switch:

- Gate LOW → Switch ON (conducts)
- Gate HIGH → Switch OFF (open)

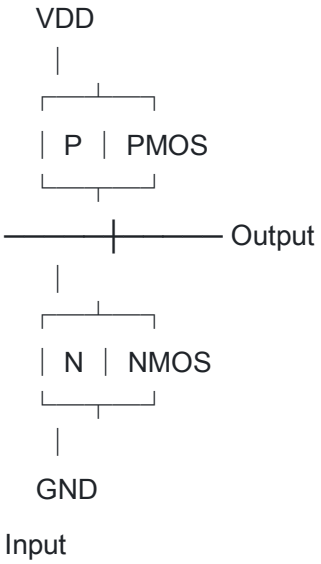
Advantages:

- High switching speed
- Low power consumption
- High impedance in OFF state
- No mechanical wear

CMOS Inverter

A CMOS inverter (NOT gate) uses complementary PMOS and NMOS transistors.

Code



Operation:

- Input HIGH: NMOS ON, PMOS OFF → Output LOW
- Input LOW: NMOS OFF, PMOS ON → Output HIGH

CMOS Logic Gates

CMOS NAND Gate:

- Two PMOS in parallel (pull-up network)

- Two NMOS in series (pull-down network)

CMOS NOR Gate:

- Two PMOS in series (pull-up network)
- Two NMOS in parallel (pull-down network)

Design principle:

- Pull-up network: Use PMOS, conducts when output should be HIGH
- Pull-down network: Use NMOS, conducts when output should be LOW

Operational Amplifier

An operational amplifier (op-amp) is a high-gain voltage amplifier with:

- Inverting input (-)
- Non-inverting input (+)
- Output

Ideal Op-amp Characteristics:

- Infinite open-loop gain
- Infinite input impedance
- Zero output impedance
- Infinite bandwidth
- Zero offset voltage

Negative Feedback in Op-amps

Negative feedback connects output to inverting input, creating stable circuits.

Benefits:

- Controlled gain
- Improved bandwidth
- Reduced distortion
- Stable operation

Closed-loop gain: ACL = -Rf/Rin (for inverting amplifier)

Basic Op-amp Applications

1. **Inverting Amplifier:**
    - Vout = -(Rf/Rin) * Vin
2. **Non-inverting Amplifier:**
    - Vout = (1 + Rf/Rin) * Vin
3. **Voltage Follower (Buffer):**
    - Vout = Vin (unity gain)
4. **Summing Amplifier:**
    - Vout = -(Rf/R1 * V1 + Rf/R2 * V2 + ...)
5. **Integrator:**
    - Vout = -(1/RC) ∫ Vin dt
6. **Differentiator:**
    - Vout = -RC * dVin/dt

---

6. PART 1 Sequential logical: Latches and Flip-Flops. Counters. Shift registers. Memories. PART 2 New elements of HTML5. New features of CSS3. Control structures in web scripts. Sensor through a web page. Providing remote management systems through a web page.

Part 1

Sequential Logic

Sequential logic circuits have outputs that depend on both current inputs and previous states. They contain memory elements that store state information.

Characteristics:

- Have feedback paths
- State depends on history
- Require clock signals (for synchronous circuits)
- Used in counters, registers, memory

Latches

Latches are basic storage elements that operate on signal levels (level-sensitive).

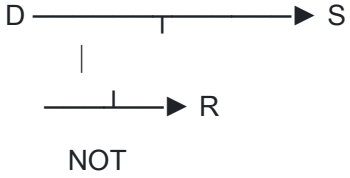
SR Latch (using NOR gates):

S	R	Q	Q'	State
0	0	Q	Q'	Hold (no change)
0	1	0	1	Reset
1	0	1	0	Set
1	1	X	X	Invalid (forbidden)

D Latch:

- Single data input D
- Eliminates invalid state
- Q follows D when enabled

```
Code
```



Flip-Flops

Flip-flops are edge-triggered storage elements that change state only on clock edges.

Types:

1. **D Flip-Flop:**
    - Q takes value of D on clock edge
    - Most common type
2. **JK Flip-Flop:**
    - J=K=1 toggles output
    - No invalid state
3. **T Flip-Flop:**
    - Toggles when T=1
    - Used in counters

D Flip-Flop Truth Table:

D	CLK↑	Q(next)
0	↑	0
1	↑	1

Counters

Counters store the count of events or clock pulses.

Types:

1. **Asynchronous (Ripple) Counter:**
    - Each flip-flop is triggered by the previous one
    - Simple but slower due to propagation delay
2. **Synchronous Counter:**
    - All flip-flops triggered by same clock
    - Faster, no ripple delay

4-bit Binary Counter:

- Counts from 0000 (0) to 1111 (15)
- Uses 4 flip-flops

Code

```
CLK ──► FF0 ──► FF1 ──► FF2 ──► FF3
      Q0     Q1     Q2     Q3
```

Shift Registers

Shift registers move data bits from one position to another on each clock pulse.

Types:

1. **SISO (Serial In, Serial Out)** – Data enters and exits serially
2. **SIPO (Serial In, Parallel Out)** – Serial input, parallel output
3. **PISO (Parallel In, Serial Out)** – Parallel input, serial output
4. **PIPO (Parallel In, Parallel Out)** – All bits loaded/read at once

Applications:

- Data serialization/deserialization
- Time delay
- Sequence generation
- Data conversion

Memories

Memory stores data for later retrieval.

Types:

1. **RAM (Random Access Memory):**
    - Read and write capability
    - Volatile (loses data without power)
    - Types: SRAM, DRAM
2. **ROM (Read-Only Memory):**
    - Read-only (written once or at factory)
    - Non-volatile

    - Types: ROM, PROM, EPROM, EEPROM
3. **Flash Memory:**
    - Electrically erasable
    - Non-volatile
    - Used in SSDs, USB drives

Part 2

New Elements of HTML5

HTML5 introduced semantic and multimedia elements:

Semantic Elements:

- <header> – Page or section header
- <nav> – Navigation links
- <main> – Main content
- <article> – Independent content
- <section> – Thematic grouping
- <aside> – Sidebar content
- <footer> – Page or section footer

Multimedia Elements:

- <audio> – Embed audio content
- <video> – Embed video content
- <canvas> – Drawing graphics via JavaScript
- <svg> – Scalable vector graphics

Form Elements:

- <datalist> – Predefined input options
- <output> – Calculation results
- <progress> – Progress indicator
- <meter> – Scalar measurement

Other:

- <figure> and <figcaption> – Images with captions
- <mark> – Highlighted text
- <time> – Date/time representation

New Features of CSS3

1. **Border Radius:**

```CSS
border-radius: 10px;
```

2. **Box Shadow:**

```CSS
box-shadow: 5px 5px 10px rgba(0,0,0,0.5);
```

3. **Gradients:**

```CSS
background: linear-gradient(to right, red, blue);
```

4. **Transitions:**

```CSS
transition: all 0.3s ease;
```

5. **Animations:**

```CSS
@keyframes fadeIn {
  from { opacity: 0; }
  to { opacity: 1; }
}
```

6. **Flexbox:**

```CSS
display: flex;
justify-content: center;
```

7. **Grid Layout:**

```CSS
display: grid;
grid-template-columns:  1fr 2fr 1fr;
```

8. **Media Queries:**

```CSS
@media (max-width: 768px) {
    /* Mobile styles */
}
```

Control Structures in Web Scripts

JavaScript Control Structures:

1. **Conditional (if-else):**

```JavaScript
if (condition) {
    // true block
} else {
    // false block
}
```

2. **Switch:**

```JavaScript
switch (value) {
    case 1:  /* code */; break;
    case 2: /* code */; break;
    default: /* code */;
}
```

3. **Loops:**

```JavaScript
for (let i = 0; i < 10; i++) { }
while (condition) { }
do { } while (condition);
for (let item of array) { }
```

4. **Exception Handling:**

```JavaScript
try {
    // risky code
} catch (error) {
    // handle error
} finally {
    // cleanup
}
```

Sensors Through Web Pages

The Generic Sensor API provides access to device sensors:

Available Sensors:

- Accelerometer
- Gyroscope
- Magnetometer
- Ambient Light Sensor
- Geolocation

```JavaScript
// Accelerometer example
const sensor = new Accelerometer();
sensor.addEventListener('reading', () => {
    console.log(sensor.x, sensor.y, sensor. z);
});
sensor.start();
```

Remote Management Through Web Pages

Remote management allows controlling systems via web interfaces:

Technologies:

1. **WebSocket** – Real-time bidirectional communication
2. **REST APIs** – HTTP-based control interfaces
3. **AJAX** – Asynchronous updates without page reload

Applications:

- IoT device control
- Server administration panels
- Network device configuration
- Home automation

---

7. PART 1 Basics of software and hardware testing, basic testing methodologies, test levels, unit testing through examples from advanced programming languages and/or hardware description languages. PART 2 Implementation of control structures in assembly language (conditional and unconditional control transfer, branching, cycle organization, subroutine call).

Part 1

Software and Hardware Testing Basics

Testing is the process of evaluating a system to find defects and verify that it meets requirements.

Objectives:

- Find defects before deployment
- Verify functionality meets specifications
- Validate system meets user needs

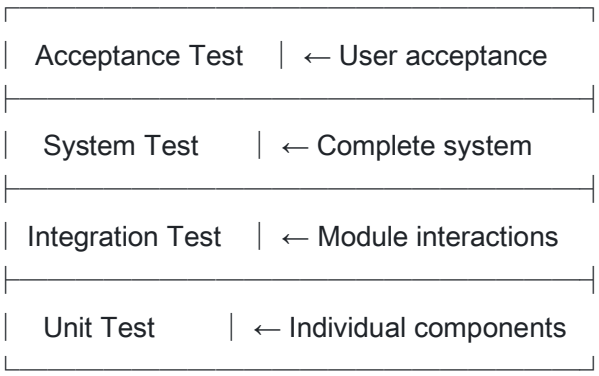
- Ensure quality and reliability

Testing Methodologies

1. **Black-Box Testing:**
    - Tests functionality without knowing internal structure
    - Based on requirements and specifications
    - Examples: Equivalence partitioning, boundary value analysis
2. **White-Box Testing:**
    - Tests internal structures and code
    - Requires knowledge of implementation
    - Examples: Statement coverage, branch coverage, path testing
3. **Gray-Box Testing:**
    - Combination of black-box and white-box
    - Partial knowledge of internals

Test Levels

Code



1. **Unit Testing:**
    - Tests individual components/functions
    - Written by developers
    - Fast and automated
2. **Integration Testing:**
    - Tests interactions between modules
    - Detects interface defects

3. **System Testing:**
    - Tests complete system
    - Verifies end-to-end functionality
4. **Acceptance Testing:**
    - Validates system meets user requirements
    - Often performed by users

Unit Testing Examples

Python (pytest):

```Python
def add(a, b):
    return a + b


def test_add():
    assert add(2, 3) == 5
    assert add(-1, 1) == 0
    assert add(0, 0) == 0
```

Java (JUnit):

```Java
import org.junit.Test;
import static org.junit.Assert.*;


public class CalculatorTest {
    @Test
    public void testAdd() {
        Calculator calc = new Calculator();
        assertEquals(5, calc.add(2, 3));
    }
}
```

Verilog (Hardware Testing):

```Verilog
module adder_tb;
    reg [3:0] a, b;
```

```
    wire [4:0] sum;

    adder uut (. a(a), .b(b), .sum(sum));

    initial begin
        a = 4'b0010; b = 4'b0011;
        #10;
        if (sum !== 5'b00101)
            $display("Test FAILED");
        else
            $display("Test PASSED");
    end
endmodule
```

Part 2

Control Structures in Assembly Language

Assembly language implements control structures using jump instructions and flags.

Sequential Execution

Instructions execute in order by default:

```
Assembly
MOV EAX, 5      ; EAX = 5
ADD EAX, 3      ; EAX = 8
MOV EBX, EAX    ; EBX = 8
```

Unconditional Control Transfer

JMP (Jump):

```
Assembly
    JMP label    ; Always jump to label
    ; code here is skipped
```

```
label:
    ; execution continues here
```

Conditional Branching

Compare and Jump:

```Assembly
    CMP EAX, EBX     ; Compare EAX with EBX
    JE  equal        ; Jump if Equal
    JNE not_equal    ; Jump if Not Equal
    JG  greater      ; Jump if Greater
    JL  less         ; Jump if Less
    JGE ge           ; Jump if Greater or Equal
    JLE le           ; Jump if Less or Equal
```

If-Else Implementation:

```Assembly
; if (EAX > 5) { EBX = 1; } else { EBX = 0; }
    CMP EAX, 5
    JG  if_true
    MOV EBX, 0       ; else branch
    JMP endif
if_true:
    MOV EBX, 1       ; if branch
endif:
    ; continue
```

Cycle Organization (Loops)

While Loop:

```Assembly
; while (ECX > 0) { ECX--; }
while_start:
    CMP ECX, 0
    JLE while_end     ; exit if ECX <= 0
    DEC ECX
```

```
    JMP while_start
while_end:
```

For Loop:

```
Assembly
; for (int i = 0; i < 10; i++)
    MOV ECX, 0      ; i = 0
for_start:
    CMP ECX, 10
    JGE for_end     ; exit if i >= 10
    ; loop body here
    INC ECX         ; i++
    JMP for_start
for_end:
```

LOOP Instruction:

```
Assembly
    MOV ECX, 10     ; counter
loop_start:
    ; loop body
    LOOP loop_start   ; decrement ECX, jump if not zero
```

Subroutine Call

CALL and RET:

```
Assembly
; Main program
    PUSH 5          ; Push argument
    CALL my_function  ; Call subroutine
    ADD ESP, 4      ; Clean up stack

; Subroutine
my_function:
    PUSH EBP        ; Save base pointer
    MOV EBP, ESP    ; Set up stack frame
```

```
    MOV EAX, [EBP+8]  ; Get argument
    ; function body

    MOV ESP, EBP      ; Restore stack
    POP EBP           ; Restore base pointer
    RET               ; Return to caller
```

Stack Frame:

```
Code
        ┌──────────────────────┐
ESP ──► │  Local vars          │
        ├──────────────────────┤
EBP ──► │  Saved EBP           │
        ├──────────────────────┤
        │  Return addr         │
        ├──────────────────────┤
        │  Arguments           │
        └──────────────────────┘
```

---

8. PART 1 Typical peripherals and communication protocols for embedded systems. Describe the control unit, peripherals, and applicability of a single-board mini-computer in embedded systems. PART 2 Describe the functions and services of network management systems and the possibilities of implementing these

functions for specific products (for example MRTG or Nagios).

Part 1

Typical Peripherals for Embedded Systems

1. **Communication Interfaces:**
    - UART/USART – Serial communication
    - SPI – High-speed synchronous serial
    - I2C – Two-wire multi-device bus
    - CAN – Automotive networks
    - USB – Universal connectivity
2. **Input Devices:**
    - GPIO – Digital input pins
    - ADC – Analog-to-digital converter
    - Buttons, keypads
    - Touch sensors
    - Various sensors (temperature, pressure, motion)
3. **Output Devices:**
    - GPIO – Digital output pins
    - DAC – Digital-to-analog converter
    - PWM – Pulse width modulation
    - LEDs, displays
    - Motors, actuators
4. **Storage:**
    - SD/MMC cards
    - EEPROM
    - Flash memory
5. **Networking:**
    - Ethernet

- Wi-Fi modules
- Bluetooth
- LoRa, Zigbee

Communication Protocols

Inter-System Protocols:

Protocol	Description	Speed	Distance
UART	Asynchronous serial	Up to 1 Mbps	Short
USB	Universal serial bus	Up to 480 Mbps	Short
Ethernet	Network protocol	10/100/1000 Mbps	Medium

Intra-System Protocols:

Protocol	Description	Wires	Features
I2C	Two-wire bus	2 (SDA, SCL)	Multi-master, addressed
SPI	Synchronous serial	4 (MOSI, MISO, SCK, SS)	Fast, full duplex
CAN	Controller Area Network	2	Robust, automotive

Single-Board Computers (SBC)

Single-board computers integrate all components on one circuit board.

Examples:

- Raspberry Pi
- BeagleBone
- Arduino (microcontroller)
- ESP32

Components:

1. **Processor** – ARM, x86, or custom
2. **Memory** – RAM and flash storage
3. **I/O** – GPIO, USB, HDMI, audio
4. **Networking** – Ethernet, Wi-Fi, Bluetooth
5. **Power management** – Voltage regulators

Applications in Embedded Systems:

- IoT gateways
- Home automation controllers
- Educational platforms
- Prototyping and development
- Industrial control systems
- Media centers

Part 2

Network Management Systems

Network management systems monitor, control, and optimize network infrastructure.

FCAPS Framework:

1. **Fault Management:**
    - Detect, isolate, and correct faults
    - Alert on failures
    - Log error events
2. **Configuration Management:**
    - Track device configurations
    - Deploy configuration changes
    - Maintain consistency
3. **Accounting Management:**
    - Track resource usage

    - Billing and cost allocation
    - Usage statistics

4. **Performance Management:**
    - Monitor network metrics
    - Identify bottlenecks
    - Trend analysis

5. **Security Management:**
    - Access control
    - Intrusion detection
    - Security policy enforcement

MRTG (Multi Router Traffic Grapher)

MRTG is a free tool for monitoring network traffic.

Features:

- Monitors SNMP-enabled devices
- Generates HTML pages with traffic graphs
- Shows daily, weekly, monthly, and yearly trends
- Uses Perl and generates PNG images

Configuration:

```
Code
Target[router]: 2: public@192.168.1.1
MaxBytes[router]: 125000000
Title[router]: Traffic on Router
PageTop[router]: <h1>Router Traffic</h1>
```

Output:

- Real-time traffic graphs
- Historical data visualization
- Bandwidth utilization reports

Nagios

Nagios is an open-source monitoring and alerting system.

Features:

- Monitors hosts, services, and network protocols
- Flexible notification system
- Web-based interface
- Plugin architecture
- Event handlers for automatic problem resolution

Components:

1. **Core** – Scheduling and event processing
2. **Plugins** – Check scripts for various services
3. **Web Interface** – Dashboard and configuration
4. **NRPE** – Remote plugin executor

Monitoring Capabilities:

- Host availability (ping)
- Service status (HTTP, SMTP, SSH)
- Resource usage (CPU, memory, disk)
- SNMP data collection
- Custom checks via plugins

Configuration Example:

Code

```
define host {
    host_name       webserver
    address         192.168.1.100
    check_command   check-host-alive
}

define service {
    host_name       webserver
    service_description HTTP
    check_command   check_http
}
```

---

9. PART 1 Programmable logic devices. Designing a digital system in hardware description language, and implementing it in FPGA devices. PART 2 Basic concepts of system engineering, different paradigms. Characteristics of the classical methods: waterfall, evolution, incremental, agile methods. Fundamentals and patterns of OO design. MVC.

Part 1

Programmable Logic Devices (PLD)

PLDs are electronic components that can be configured to implement digital circuits.

Types:

1. **SPLD (Simple PLD):**
    - PAL (Programmable Array Logic)
    - PLA (Programmable Logic Array)
    - Limited complexity
2. **CPLD (Complex PLD):**
    - Multiple logic blocks
    - Non-volatile configuration
    - Predictable timing
3. **FPGA (Field-Programmable Gate Array):**

- Large array of configurable logic blocks
- Flexible routing
- Can implement complex systems

Hardware Description Languages (HDL)

HDLs describe digital circuit structure and behavior.

Main HDLs:

- **Verilog** – C-like syntax, industry standard
- **VHDL** – Ada-like syntax, verbose but precise

Verilog Example (4-bit Counter):

```Verilog
module counter(
    input clk,
    input reset,
    output reg [3:0] count
);
    always @(posedge clk or posedge reset) begin
        if (reset)
            count <= 4'b0000;
        else
            count <= count + 1;
    end
endmodule
```

VHDL Example (4-bit Counter):

```VHDL
entity counter is
    port (
        clk   : in std_logic;
        reset : in std_logic;
        count : out std_logic_vector(3 downto 0)
    );
end counter;
```

```
architecture behavioral of counter is
    signal cnt : unsigned(3 downto 0);
begin
    process(clk, reset)
    begin
        if reset = '1' then
            cnt <= (others => '0');
        elsif rising_edge(clk) then
            cnt <= cnt + 1;
        end if;
    end process;
    count <= std_logic_vector(cnt);
end behavioral;
```

FPGA Implementation

FPGA Architecture:

- **CLB (Configurable Logic Block)** – Contains LUTs and flip-flops
- **IOB (Input/Output Block)** – Interface with external pins
- **Routing fabric** – Programmable interconnects
- **Block RAM** – On-chip memory
- **DSP blocks** – Dedicated arithmetic units

Design Flow:

1. Design entry (HDL or schematic)
2. Synthesis – Convert to gate-level netlist
3. Implementation – Place and route
4. Generate bitstream
5. Program FPGA

Part 2

System Engineering

System engineering is an interdisciplinary approach to designing and managing complex systems throughout their life cycle.

Key Concepts:

1. **Value** – Systems provide value when meeting stakeholder needs
2. **Context** – Consider the environment where the system operates
3. **Trade-offs** – Balance cost, time, and performance
4. **Abstraction** – Design independently of specific solutions
5. **Interdisciplinarity** – Teams from various disciplines

Software Development Paradigms

1. Waterfall Model:

- Linear, sequential phases
- Each phase must complete before the next begins
- Phases: Requirements → Design → Implementation → Testing → Deployment → Maintenance

Advantages: Simple, well-documented *Disadvantages: * Inflexible, no working software until late

2. Evolutionary Model:

- Initial version developed, then refined
- Continuous improvement based on feedback
- Good for systems with unclear requirements

3. Incremental Model:

- System divided into increments
- Each increment adds functionality
- Partial system delivered early

4. Agile Methods:

- Iterative and incremental
- Emphasis on collaboration and adaptability
- Short iterations (sprints)
- Working software over documentation

Agile Principles:

- Customer collaboration
- Responding to change
- Working software
- Individuals and interactions

Object-Oriented Design Fundamentals

Four Pillars:

1. **Abstraction** – Hide implementation details, expose interface
2. **Encapsulation** – Bundle data and methods, hide internals
3. **Inheritance** – Create new classes from existing ones
4. **Polymorphism** – Same interface, different implementations

Design Patterns

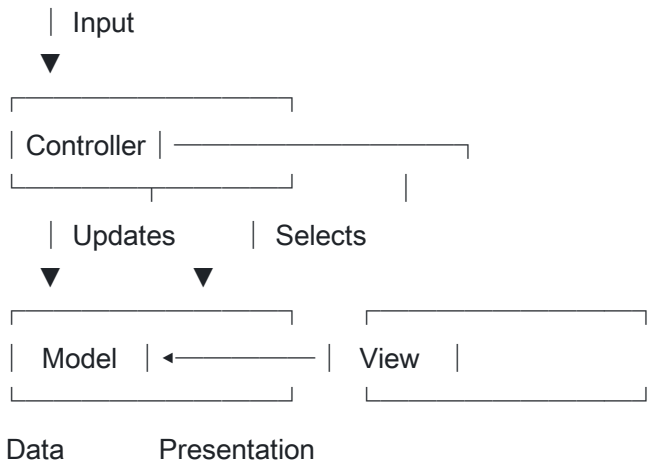
Categories:

1. **Creational Patterns:**
    - Factory, Abstract Factory, Singleton, Builder
    - Control object creation
2. **Structural Patterns:**
    - Adapter, Decorator, Facade, Composite
    - Organize class relationships
3. **Behavioral Patterns:**
    - Observer, Strategy, Command, Iterator
    - Manage object communication

MVC Architecture

Model-View-Controller separates concerns in application design.

Code



Components:

- **Model** – Data and business logic
- **View** – User interface/presentation
- **Controller** – Handles input, updates model and view

Benefits:

- Separation of concerns
- Parallel development
- Easier testing
- Code reusability

---

10. PART 1 Web server configuration using SSL, the OpenSSL cryptographic library basic functions: authentication, encryption. PART 2 The instruction set architecture (ISA) of Intel X86 processors (registers, addressing, instructions, memory architecture, interrupt system).

Part 1

SSL/TLS

SSL (Secure Sockets Layer) and its successor TLS (Transport Layer Security) provide secure communication over networks.

Security Features:

1. **Encryption** – Data privacy
2. **Authentication** – Identity verification
3. **Integrity** – Detect tampering

Web Server SSL Configuration

Apache Configuration:

```apache
<VirtualHost *:443>
    ServerName www.example.com
    DocumentRoot /var/www/html

    SSLEngine on
    SSLCertificateFile /etc/ssl/certs/server.crt
    SSLCertificateKeyFile /etc/ssl/private/server.key
    SSLCertificateChainFile /etc/ssl/certs/ca-bundle. crt

    SSLProtocol all -SSLv3 -TLSv1 -TLSv1.1
    SSLCipherSuite HIGH:!aNULL:!MD5
</VirtualHost>
```

Nginx Configuration:

```Nginx
server {
    listen 443 ssl;
    server_name www.example.com;

    ssl_certificate /etc/ssl/certs/server.crt;
    ssl_certificate_key /etc/ssl/private/server.key;
```

```
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;
}
```

OpenSSL

OpenSSL is an open-source cryptographic library.

Key Generation:

```
bash
# Generate private key
openssl genrsa -out private.key 2048

# Generate certificate signing request
openssl req -new -key private.key -out
```

Computer Science Engineering BSc (2021) – Final Exam Answers (Continued)

Continuing from where we left off with Questions 10-15:

---

10. PART 1 Web server configuration using SSL, the OpenSSL cryptographic library basic functions: authentication, encryption. PART 2 The instruction set architecture (ISA) of Intel X86 processors (registers, addressing,

instructions, memory architecture, interrupt system).

Part 1 (Continued)

OpenSSL Commands

Key Generation:

```bash
# Generate RSA private key
openssl genrsa -out private.key 2048

# Generate certificate signing request (CSR)
openssl req -new -key private.key -out request.csr

# Generate self-signed certificate
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
    -keyout private. key -out certificate. crt

# View certificate details
openssl x509 -in certificate.crt -text -noout
```

SSL Authentication

SSL authentication verifies the identity of communicating parties.

Server Authentication (Most Common):

1. Client connects to server
2. Server presents its certificate
3. Client verifies certificate against trusted Certificate Authorities (CA)
4. If valid, secure connection is established

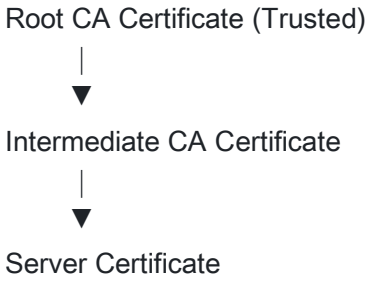
Mutual Authentication (Two-Way):

- Both client and server present certificates

- Used in high-security environments

Certificate Chain:

Code

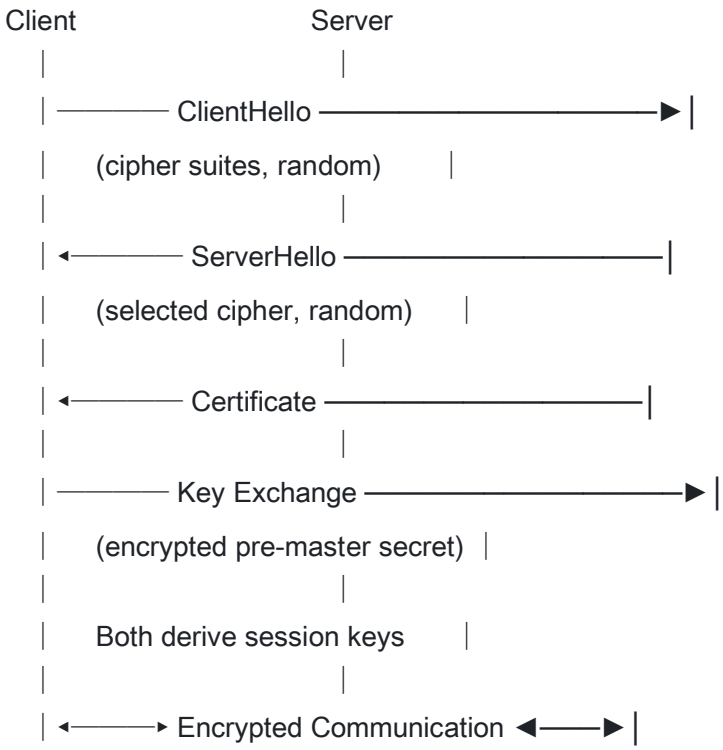


SSL Encryption

SSL uses hybrid encryption combining asymmetric and symmetric cryptography.

Handshake Process:

Code



Encryption Types:

- **Asymmetric (RSA, ECDSA):** Key exchange and authentication
- **Symmetric (AES, ChaCha20):** Bulk data encryption
- **Hashing (SHA-256):** Message integrity

Part 2

Instruction Set Architecture (ISA)

The ISA defines the interface between software and hardware, specifying how the processor executes instructions.

X86 Registers

General-Purpose Registers (32-bit):

Register	Purpose
EAX	Accumulator (arithmetic, return values)
EBX	Base register (memory addressing)
ECX	Counter (loops, string operations)
EDX	Data register (I/O, multiplication)
ESI	Source Index (string operations)
EDI	Destination Index (string operations)
EBP	Base Pointer (stack frame base)
ESP	Stack Pointer (top of stack)

Segment Registers:

- CS (Code Segment), DS (Data Segment), SS (Stack Segment)
- ES, FS, GS (Extra segments)

Special Registers:

- EIP (Instruction Pointer)
- EFLAGS (Status flags: Zero, Carry, Sign, Overflow)

X86 Addressing Modes

1. **Register Addressing:**

```
Assembly
MOV EAX, EBX      ; Copy EBX to EAX
```

2. **Immediate Addressing:**

```
Assembly
MOV EAX, 100      ; Load constant 100 into EAX
```

3. **Direct Memory Addressing:**

```
Assembly
MOV EAX, [0x1000] ; Load value at address 0x1000
```

4. **Register Indirect:**

```
Assembly
MOV EAX, [EBX]    ; Load value at address in EBX
```

5. **Base + Displacement:**

```
Assembly
MOV EAX, [EBX+8]  ; Load value at EBX+8
```

6. **Scaled Index:**

```
Assembly
MOV EAX, [EBX+ECX*4+8]  ; Array access
```

X86 Instructions

Data Movement:

- MOV – Move data
- PUSH / POP – Stack operations
- LEA – Load effective address

Arithmetic:

- ADD, SUB, MUL, DIV

- INC, DEC
- IMUL, IDIV (signed)

Logical:

- AND, OR, XOR, NOT
- SHL, SHR (shift)
- ROL, ROR (rotate)

Control Flow:

- JMP – Unconditional jump
- Jcc – Conditional jumps (JE, JNE, JG, JL, etc.)
- CALL, RET – Subroutine calls
- LOOP – Loop with counter

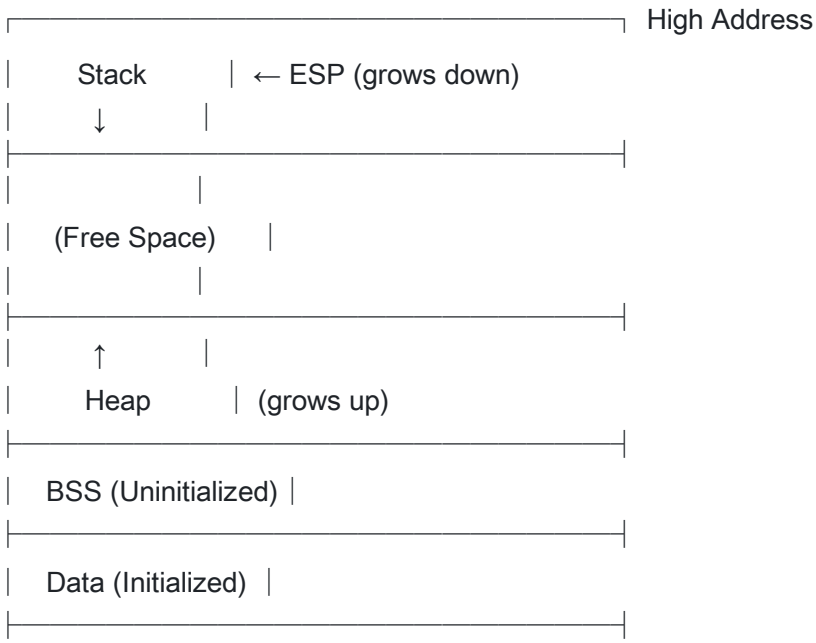
Comparison:

- CMP – Compare (sets flags)
- TEST – Logical compare

X86 Memory Architecture

Memory Model:

```
Code
```



Segmentation and Paging:

- Logical Address → Segmentation → Linear Address → Paging → Physical Address
- Protected mode uses paging for virtual memory

Interrupt System

Interrupts are signals that cause the processor to suspend current execution and handle an event.

Types of Interrupts:

1. **Hardware Interrupts:**
    - Generated by external devices (keyboard, timer)
    - Asynchronous to program execution
    - Handled via Interrupt Request (IRQ) lines
2. **Software Interrupts:**
    - Generated by `INT` instruction
    - Used for system calls
    - Example: `INT 0x80` (Linux system call)
3. **Exceptions:**
    - Generated by CPU errors
    - Examples: Division by zero, page fault, invalid opcode

Interrupt Descriptor Table (IDT):

- 256 entries (0-255)
- Each entry points to an interrupt handler
- Loaded via `LIDT` instruction

Interrupt Handling Process:

1. Push EFLAGS, CS, EIP onto stack
2. Clear interrupt flag (if applicable)
3. Load handler address from IDT

4. Execute handler
5. IRET instruction returns from interrupt

---

11. PART 1 Inter-process communication methods (file, signal, pipe, socket). PART 2 Complexity of algorithms, asymptotic notations. Insertion sort, searching in linear and logarithmic time. Tables, hash functions, hash tables. Graphs, breadth-first and depth-first search.

Part 1

Inter-Process Communication (IPC)

IPC mechanisms allow processes to exchange data and synchronize their actions.

File-Based Communication

Files provide persistent storage that multiple processes can access.

Characteristics:

- Simple and widely supported
- Persistent data storage
- Slower than other methods
- Requires synchronization for concurrent access

Example (Python):

```
Python
# Writer process
with open('shared_file.txt', 'w') as f:
```

```
    f.write('Hello from Process A')

# Reader process
with open('shared_file.txt', 'r') as f:
    data = f.read()
```

Challenges:

- Race conditions
- File locking needed
- No real-time notification

Signals

Signals are software interrupts sent to processes to notify them of events.

Characteristics:

- Asynchronous communication
- Limited data transfer (just the signal number)
- Used for control rather than data exchange

Common Signals:

Signal	Number	Description
SIGINT	2	Interrupt (Ctrl+C)
SIGKILL	9	Force termination
SIGTERM	15	Graceful termination
SIGUSR1	10	User-defined signal 1
SIGCHLD	17	Child process status change

Example (C):

```
C
#include <signal. h>
```

```c
void handler(int sig) {
    printf("Received signal %d\n", sig);
}

int main() {
    signal(SIGUSR1, handler);
    // Process continues...
    pause();  // Wait for signal
    return 0;
}
```

Pipes

Pipes provide unidirectional data flow between processes.

Types:

1. **Anonymous Pipes:**
    - Created with pipe() system call
    - Used between parent and child processes
    - Temporary, exist only while processes run

```c
int fd[2];
pipe(fd);  // fd[0] = read end, fd[1] = write end

if (fork() == 0) {
    // Child reads
    close(fd[1]);
    read(fd[0], buffer, size);
} else {
    // Parent writes
    close(fd[0]);
    write(fd[1], "Hello", 5);
}
```

2. **Named Pipes (FIFOs):**

- Created with mkfifo()
- Persist in file system
- Allow communication between unrelated processes

```bash
mkfifo /tmp/myfifo
echo "Hello" > /tmp/myfifo &
cat < /tmp/myfifo
```

Sockets

Sockets provide bidirectional communication, both locally and over networks.

Types:

1. **Unix Domain Sockets:**
   - Local communication only
   - Use file system path as address
   - Faster than network sockets
2. **Network Sockets:**
   - TCP sockets (stream, reliable)
   - UDP sockets (datagram, unreliable)
   - Can communicate across machines

Socket Programming Flow:

```
Code
Server                  Client
   |                      |
socket()                socket()
   |                      |
bind()                    |
   |                      |
listen()                  |
   |                      |
accept() ◄──────────────── connect()
   |                      |
read()/write() ◄──────────► read()/write()
   |                      |
```

close() close()

Example (Python TCP Server):

```
Python
import socket

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(('localhost', 8080))
server.listen(1)

conn, addr = server. accept()
data = conn.recv(1024)
conn.send(b'Response')
conn.close()
```

Part 2

Algorithm Complexity

Time complexity measures how an algorithm's runtime grows with input size.

Asymptotic Notations

Big O (O) - Upper Bound:

- Worst-case complexity
- f(n) = O(g(n)) means f(n) ≤ c·g(n) for large n

Big Omega (Ω) - Lower Bound:

- Best-case complexity
- f(n) = Ω(g(n)) means f(n) ≥ c·g(n) for large n

Big Theta (Θ) - Tight Bound:

- Average-case complexity
- f(n) = Θ(g(n)) means $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$

Common Complexity Classes:

```
Code
```

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

Insertion Sort

Insertion sort builds the sorted array one element at a time.

Algorithm:

```python
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr
```

Complexity:

- Best case: O(n) - already sorted
- Average case: $O(n^2)$
- Worst case: $O(n^2)$ - reverse sorted
- Space: O(1) - in-place

Visualization:

```
Code
```

```
[5, 2, 4, 6, 1, 3]
 ↑ Insert 2:  [2, 5, 4, 6, 1, 3]
   ↑ Insert 4: [2, 4, 5, 6, 1, 3]
     ↑ Insert 6: [2, 4, 5, 6, 1, 3]
       ↑ Insert 1: [1, 2, 4, 5, 6, 3]
         ↑ Insert 3: [1, 2, 3, 4, 5, 6]
```

Linear Search

Examines each element sequentially until the target is found.

```python
def linear_search(arr, target):
    for i, val in enumerate(arr):
        if val == target:
            return i
    return -1
```

Complexity: O(n)

Binary Search (Logarithmic)

Requires sorted array; repeatedly divides search space in half.

```python
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

Complexity: O(log n)

Hash Tables

Hash tables provide O(1) average-case access using a hash function.

Components:

1. **Array** - Storage for key-value pairs
2. **Hash Function** - Maps keys to array indices
3. **Collision Resolution** - Handles multiple keys mapping to same index

Hash Function Properties:

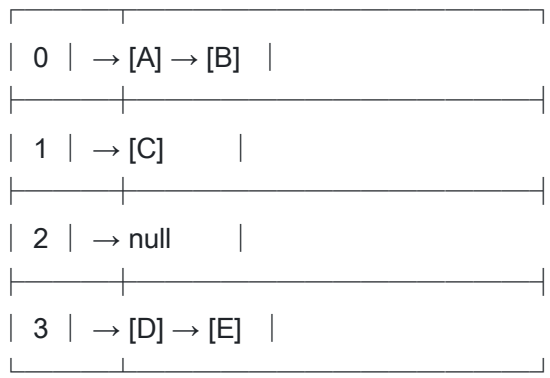
- Deterministic
- Uniform distribution
- Fast to compute

Collision Resolution:

1. **Chaining:**
    - Each slot contains a linked list
    - Multiple elements can occupy same slot
2. **Open Addressing:**
    - Linear probing: Check next slot
    - Quadratic probing: Check slot + i^2
    - Double hashing: Use second hash function

Code

Hash Table with Chaining:



Graphs

A graph G = (V, E) consists of vertices V and edges E.

Representations:

1. **Adjacency Matrix:**

Code

```
   A B C D
A [ 0 1 1 0 ]
B [ 1 0 1 1 ]
C [ 1 1 0 1 ]
```

D [0 1 1 0]

2. **Adjacency List:**

```
Code
```

A → [B, C]

B → [A, C, D]

C → [A, B, D]

D → [B, C]

Breadth-First Search (BFS)

Explores nodes level by level using a queue.

```
Python
```

```
from collections import deque

def bfs(graph, start):
    visited = set([start])
    queue = deque([start])
    result = []

    while queue:
        node = queue. popleft()
        result.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return result
```

Complexity: O(V + E)

Applications:

- Shortest path in unweighted graphs
- Level-order traversal
- Finding connected components

Depth-First Search (DFS)

Explores as deep as possible before backtracking, using recursion or stack.

```
Python
def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()
    visited.add(start)
    result = [start]

    for neighbor in graph[start]:
        if neighbor not in visited:
            result. extend(dfs(graph, neighbor, visited))
    return result
```

Complexity: O(V + E)

Applications:

- Cycle detection
- Topological sorting
- Path finding
- Detecting connected components

---

12. PART 1 Entity-relationship (ER) model, design with ER diagrams. Relational data model, relation, scheme, attribute. Building up a relational scheme from an ER-diagram. PART 2 Diodes. Rectifiers. DC to DC converters. Voltage regulators. Current regulators.

Part 1

ER Model

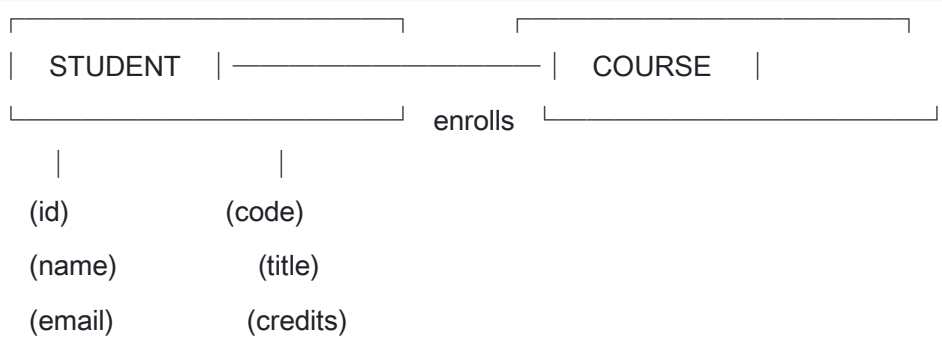
The Entity-Relationship model describes data in terms of entities, attributes, and relationships.

Components:

1. **Entity:**
    - A distinguishable object in the real world
    - Represented as rectangles in ER diagrams
    - **Strong Entity:** Has its own primary key
    - **Weak Entity:** Depends on another entity for identification
2. **Attribute:**
    - Properties of an entity
    - Represented as ovals
    - **Types:**
        - Simple vs. Composite
        - Single-valued vs. Multi-valued
        - Stored vs. Derived
3. **Relationship:**
    - Association between entities
    - Represented as diamonds
    - Has cardinality (1:1, 1:N, M:N)

ER Diagram Notation

Code



Cardinality Notations:

Notation	Meaning
1:1	One-to-one
1:N	One-to-many
M:N	Many-to-many

Relational Data Model

Key Concepts:

1. **Relation (Table):**
    - A set of tuples (rows)
    - Each tuple represents an entity instance
2. **Schema:**
    - Structure definition of a relation
    - $R(A_1, A_2, ..., A_n)$
3. **Attribute:**
    - A column in the relation
    - Has a name and domain (data type)
4. **Keys:**
    - **Super Key:** Set of attributes that uniquely identifies tuples
    - **Candidate Key:** Minimal super key
    - **Primary Key:** Chosen candidate key
    - **Foreign Key:** References primary key of another table

Converting ER to Relational Schema

Rules:

1. **Strong Entity → Table**
    - Each attribute becomes a column
    - Primary key is identified

```
STUDENT(student_id PK, name, email, date_of_birth)
```

2. **Weak Entity → Table**
    - Include foreign key from strong entity
    - Primary key is composite (FK + partial key)

```
DEPENDENT(employee_id FK, dependent_name, relationship)
PK: (employee_id, dependent_name)
```

3. **Relationships:**

Binary 1:1:

- Add foreign key to either table

```
EMPLOYEE(emp_id PK, name, dept_id FK)
DEPARTMENT(dept_id PK, name)
```

Binary 1:N:

- Add foreign key to the "many" side

```
DEPARTMENT(dept_id PK, name)
EMPLOYEE(emp_id PK, name, dept_id FK)
```

Binary M:N:

- Create a junction table

```
STUDENT(student_id PK, name)
COURSE(course_id PK, title)
ENROLLMENT(student_id FK, course_id FK, grade)
PK: (student_id, course_id)
```

Ternary Relationship:

- Create new table with foreign keys from all participants

Part 2

Diodes

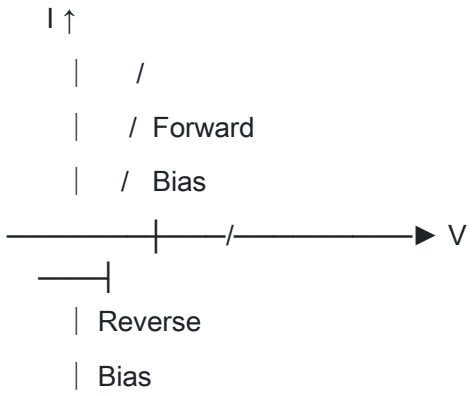
A diode is a semiconductor device that allows current to flow in one direction only.

Characteristics:

- **Anode (+)** and **Cathode (-)**
- Forward bias: Conducts when V_anode > V_cathode + V_f
- Reverse bias: Blocks current (until breakdown)
- Forward voltage drop (V_f): ~0.7V for silicon, ~0.3V for germanium

I-V Characteristic:

```
Code
```



Types:

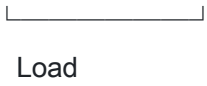
- Rectifier diodes
- Zener diodes (voltage reference)
- LEDs (light emission)
- Schottky diodes (low forward drop)
- Photodiodes (light detection)

Rectifiers

Rectifiers convert AC to DC using diodes.

Half-Wave Rectifier:

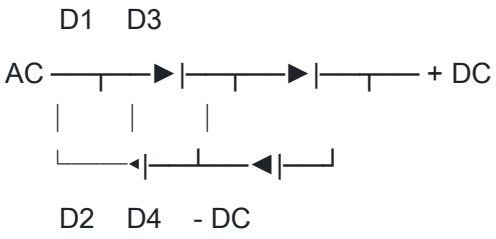
```
Code
```



- Uses one diode
- Conducts only on positive half-cycles
- Output: Pulsating DC
- Efficiency: Low (~40. 6%)

Full-Wave Rectifier (Bridge):

Code



- Uses four diodes
- Both half-cycles converted
- Higher efficiency (~81. 2%)
- Smoother output

Filtering:

- Capacitor smooths pulsating DC
- Larger capacitor = less ripple

DC-DC Converters

DC-DC converters change one DC voltage level to another.

Types:

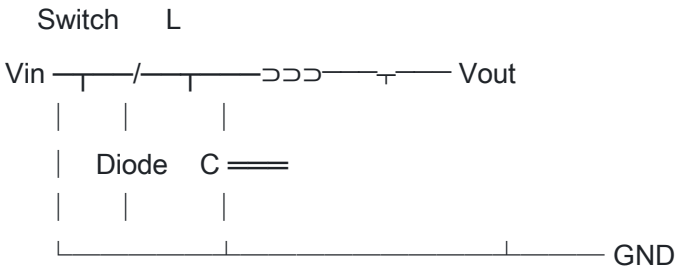
1. **Buck Converter (Step-Down):**
    - Output voltage < Input voltage
    - High efficiency
    - Uses inductor, diode, switch, capacitor
2. **Boost Converter (Step-Up):**
    - Output voltage > Input voltage

- Energy stored in inductor during switch ON
- Transferred to output during switch OFF

3. **Buck-Boost Converter:**
    - Can step up or step down
    - Output polarity inverted

Basic Buck Converter:

Code



Voltage Regulators

Voltage regulators maintain a constant output voltage despite input or load variations.

Types:

1. **Linear Regulators:**
    - Simple, low noise
    - Less efficient (excess power dissipated as heat)
    - Example: LM7805 (5V output)

Code

2. **Switching Regulators:**
    - Higher efficiency
    - More complex
    - May introduce noise
    - Used in battery-powered devices

Key Parameters:

- Line regulation: Output change vs. input change
- Load regulation: Output change vs. load change
- Dropout voltage: Minimum Vin - Vout
- Quiescent current: Current consumed by regulator

Current Regulators

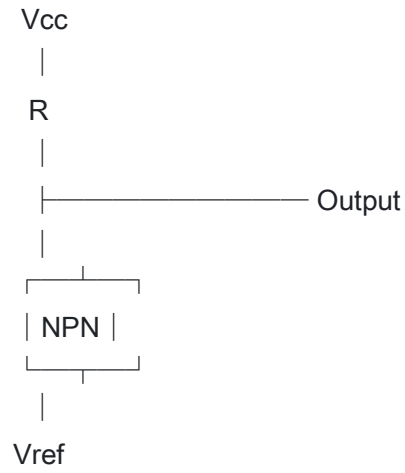
Current regulators maintain constant current regardless of load resistance.

Applications:

- LED driving
- Battery charging
- Sensor excitation

Simple Current Source (Transistor-Based):

Code



I_out ≈ (Vref - V_be) / R

Integrated Current Regulators:

- LM317 in constant current mode
- Dedicated LED driver ICs
- Programmable current sources

---

13. PART 1 Modern processor solutions (pipeline, hazard, out-of-order execution, speculative execution, superscalar-, VLIW- and vector processors) PART 2 Optimization and evaluation of relational queries. Tree-based optimization in relational algebra. Cost-based optimization.

Part 1

Pipeline

Pipelining overlaps instruction execution to increase throughput.

Classic 5-Stage Pipeline:

```
Code
Stage 1: IF  (Instruction Fetch)
Stage 2: ID  (Instruction Decode)
Stage 3: EX  (Execute)
Stage 4: MEM (Memory Access)
Stage 5: WB  (Write Back)
```

Pipeline Operation:

```
Code
Time →   1   2   3   4   5   6   7   8
Inst 1:  IF  ID  EX  MEM WB
Inst 2:      IF  ID  EX  MEM WB
Inst 3:          IF  ID  EX  MEM WB
Inst 4:              IF  ID  EX  MEM WB
```

Benefits:

- Increased throughput (one instruction completes per cycle ideally)
- Better hardware utilization

Pipeline Hazards

Hazards are situations that prevent the next instruction from executing.

1. Data Hazards: Dependencies between instructions

```
Assembly
ADD R1, R2, R3    ; R1 = R2 + R3
SUB R4, R1, R5    ; Needs R1 from previous instruction
```

Solutions:

- Forwarding/Bypassing: Route result directly to next instruction
- Stalling: Insert pipeline bubbles
- Compiler scheduling: Reorder instructions

2. Structural Hazards: Multiple instructions need the same hardware resource

Solutions:

- Duplicate resources
- Stalling

3. Control Hazards: Branch instructions change program flow

```
Assembly
BEQ R1, R2, target  ; Branch if equal
ADD R3, R4, R5      ; Should this execute?
```

Solutions:

- Branch prediction
- Delayed branching
- Speculative execution

Out-of-Order Execution

Instructions execute when operands are ready, not in program order.

Process:

1. Fetch instructions in order

2. Decode and place in reservation stations
3. Execute when operands available (out-of-order)
4. Reorder buffer ensures in-order commit

Benefits:

- Hides memory latency
- Better utilization of execution units
- Increases instruction-level parallelism

Speculative Execution

Execute instructions before knowing if they're needed.

Branch Prediction:

- Static: Always predict taken/not taken
- Dynamic: Based on history (branch history table)

Prediction Accuracy:

- Modern processors: >95% accuracy
- Misprediction penalty: Pipeline flush

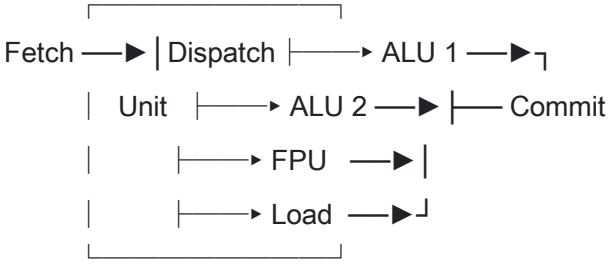
Superscalar Processors

Issue and execute multiple instructions per cycle.

Characteristics:

- Multiple execution units (ALUs, FPUs)
- Multiple instruction issue per cycle
- Hardware detects parallelism dynamically

Code



VLIW (Very Long Instruction Word)

Compiler bundles multiple operations into one long instruction.

Characteristics:

- Compiler determines parallelism (static scheduling)
- Simpler hardware than superscalar
- Large instruction words (128-1024 bits)
- Performance depends on compiler quality

Example:

```
Code
[ADD R1,R2,R3 | MUL R4,R5,R6 | LOAD R7,[R8] | NOP]
  ALU 1      ALU 2      Memory     FPU
```

Advantages:

- Simpler hardware
- Lower power consumption

Disadvantages:

- Code compatibility issues
- NOP padding wastes memory
- Compiler complexity

Vector Processors

Process arrays of data with single instructions (SIMD).

Characteristics:

- Vector registers hold multiple data elements
- Single instruction operates on entire vector
- Excellent for scientific computing, graphics

Example:

```
Code
; Scalar:  4 instructions
ADD R1, R2, R3
```

```
ADD R4, R5, R6
ADD R7, R8, R9
ADD R10, R11, R12

; Vector: 1 instruction
VADD V1, V2, V3  ; Adds 4 (or more) elements at once
```

Modern Implementations:

- SSE, AVX (x86)
- NEON (ARM)
- GPU shaders

Part 2

Query Optimization

Query optimization transforms a SQL query into an efficient execution plan.

Query Processing Steps:

1. Parsing → Syntax tree
2. Translation → Relational algebra
3. Optimization → Best execution plan
4. Execution → Results

Relational Algebra Operations

Operation	Symbol	Description
Selection	σ	Filter rows
Projection	π	Select columns
Join	⋈	Combine relations
Union	∪	Combine tuples

Operation	Symbol	Description
Difference	-	Remove tuples
Cartesian Product	×	All combinations

Tree-Based Optimization

Query trees represent the execution order of relational algebra operations.

Optimization Techniques:

1. **Push Selections Down:**
   - Apply selection as early as possible
   - Reduces intermediate result sizes

Code

```
Before:          After:
   π               π
   |               |
   σ               ⋈
   |              / \
   ⋈            σ   σ
  / \           |   |
 R   S          R   S
```

2. **Push Projections Down:**
   - Remove unnecessary attributes early
   - Reduces tuple width
3. **Combine Selections:**
   - σ_c1(σ_c2(R)) = σ_c1∧c2(R)
4. **Join Ordering:**
   - Order joins to minimize intermediate results
   - Most selective joins first

Equivalence Rules:

- σ_c1∧c2(R) = σ_c1(σ_c2(R))

- $\pi_L(\sigma_c(R)) = \sigma_c(\pi_L(R))$ if c uses only L
- $R \bowtie S = S \bowtie R$ (commutativity)
- $(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$ (associativity)

Cost-Based Optimization

Assigns costs to different execution plans and chooses the cheapest.

Cost Factors:

- Disk I/O (dominant factor)
- CPU time
- Memory usage
- Network transfer (distributed)

Statistics Used:

- Table size (number of tuples)
- Attribute cardinality (distinct values)
- Index information
- Data distribution (histograms)

Cost Estimation:

```
Code
Cost(sequential scan) = Number of pages
Cost(index scan) = Height of index + matching pages
Cost(join) = Depends on algorithm


Nested Loop: O(n × m)
Sort-Merge: O(n log n + m log m)
Hash Join: O(n + m)
```

Optimization Process:

1. Generate alternative plans
2. Estimate cost of each plan
3. Choose plan with lowest cost
4. Execute chosen plan

---

14. PART 1 Purpose, operating algorithms, similarities and differences of NAT/PAT address exchange mechanisms. PART 2 Basic notions concerning data structures: abstraction, abstract data types. Elementary data structures: lists, stacks, queues. Sets, multisets, arrays. Representation of trees, tree traversal, search, insert, delete.

Part 1

Network Address Translation (NAT)

NAT translates private IP addresses to public IP addresses, enabling multiple devices to share one public IP.

Purpose:

- Conserve IPv4 addresses
- Hide internal network structure
- Provide basic security

Types of NAT:

1. Static NAT (SNAT):

- One-to-one mapping
- Private IP permanently mapped to public IP
- Used for servers needing external access

```
Code
Private IP          Public IP
```

192.168.1.10 ←→ 203.0.113.10

192.168.1.11 ←→ 203.0.113.11

2. Dynamic NAT (DNAT):

- Pool of public addresses
- Temporary mappings from pool
- Mapping released after timeout

```
Code
Private IPs        Public IP Pool
192.168.1.10 →     [203.0.113.10]
192.168.1.11 →     [203.0.113.11]
192.168.1.12 →     [203.0.113.12]
```

Port Address Translation (PAT)

Also called NAT Overload or NAPT (Network Address Port Translation).

Operation:

- Many private IPs map to one public IP
- Distinguished by port numbers
- Most common NAT type for home/office

```
Code
NAT Table:
┌──────────────────┬──────────────────┬──────────────┐
│ Private Address  │ Public Address   │ Port Mapping │
├──────────────────┼──────────────────┼──────────────┤
│ 192.168.1.10:1234│ 203.0.113.1:5001 │ TCP          │
│ 192.168.1.11:1234│ 203.0.113.1:5002 │ TCP          │
│ 192.168.1.12:4567│ 203.0.113.1:5003 │ UDP          │
└──────────────────┴──────────────────┴──────────────┘
```

PAT Process:

1. Internal host sends packet
2. Router replaces source IP with public IP
3. Router assigns unique source port

4. Records mapping in NAT table
5. Response packets are translated back

NAT vs PAT Comparison

Feature	NAT	PAT
Address Mapping	One-to-one (static) or pool (dynamic)	Many-to-one
Port Translation	No	Yes
Public IPs Needed	Multiple	One
Scalability	Limited by public IPs	High (65,535 ports)
Cost	Higher (more public IPs)	Lower
Complexity	Simple	More complex

Similarities

- Both translate private to public addresses
- Both provide address conservation
- Both hide internal network
- Both maintain translation tables
- Both work at Layer 3 (Network)

Differences

- NAT uses IP addresses only; PAT uses IP + port
- NAT requires more public IPs; PAT needs only one
- PAT more complex but more scalable
- NAT better for servers; PAT for client networks

Part 2

Data Structure Concepts

Abstraction:

- Hiding implementation details
- Exposing only essential features
- Separation of "what" from "how"

Abstract Data Type (ADT):

- Mathematical model for data types
- Defines operations, not implementation
- Examples: Stack, Queue, List, Set

ADT Components:

1. Data domain (possible values)
2. Operations (functions)
3. Axioms (behavior rules)

Lists

A list is a linear collection of elements with order.

Types:

1. Array-based List:

Code

2. Linked List:

Code

Linked List Types:

- Singly linked (one pointer)
- Doubly linked (two pointers)

- Circular (last points to first)

Operations and Complexity:

Operation	Array	Linked List
Access by index	O(1)	O(n)
Insert at front	O(n)	O(1)
Insert at end	O(1)*	O(n) or O(1)**
Delete	O(n)	O(1)***
Search	O(n)	O(n)

*Amortized **With tail pointer ***Given node reference

Stacks

LIFO (Last In, First Out) structure.

```
Code
    |   |
    | C | ← Top (push/pop here)
    | B |
    | A |
    └───┘
```

Operations:

- `push(item)` - Add to top, O(1)
- `pop()` - Remove from top, O(1)
- `peek()` - View top, O(1)
- `isEmpty()` - Check if empty, O(1)

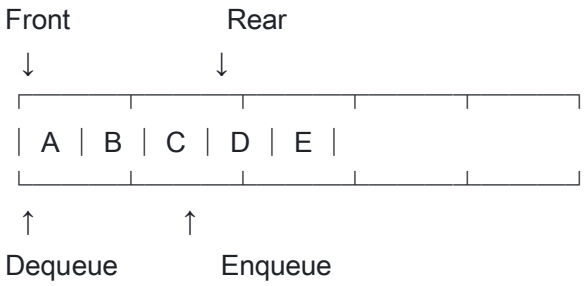
Applications:

- Function call stack
- Expression evaluation
- Undo operations
- Backtracking algorithms

Queues

FIFO (First In, First Out) structure.

Code



Operations:

- enqueue(item) - Add to rear, O(1)
- dequeue() - Remove from front, O(1)
- front() - View front, O(1)
- isEmpty() - Check if empty, O(1)

Variants:

- Circular Queue
- Priority Queue
- Deque (double-ended)

Sets and Multisets

Set:

- Unordered collection of unique elements
- No duplicates allowed
- Operations: union, intersection, difference

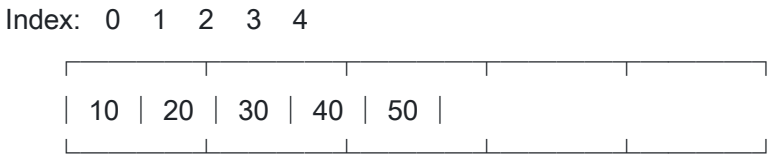
Multiset (Bag):

- Allows duplicate elements
- Tracks count of each element

Arrays

Fixed-size sequential collection with direct access.

Code



Address = Base + Index × Size

Properties:

- Constant-time access O(1)
- Fixed size (static arrays)
- Contiguous memory

Trees

Hierarchical data structure with nodes and edges.

Terminology:

- Root: Top node
- Parent/Child: Adjacent nodes
- Leaf: Node with no children
- Height: Longest path from root to leaf
- Depth: Distance from root

Binary Tree:

```
Code
```

```
    A       ← Root
   / \
  B   C
 / \   \
D   E   F   ← Leaves
```

Binary Search Tree (BST):

- Left subtree < Node < Right subtree
- Enables efficient search

Tree Traversal

1. Pre-order (Root, Left, Right):

```
Python
```

```
def preorder(node):
    if node:
        visit(node)
        preorder(node.left)
        preorder(node.right)
```

Result: A, B, D, E, C, F

2. In-order (Left, Root, Right):

```
Python
def inorder(node):
    if node:
        inorder(node.left)
        visit(node)
        inorder(node.right)
```

Result: D, B, E, A, C, F (sorted for BST)

3. Post-order (Left, Right, Root):

```
Python
def postorder(node):
    if node:
        postorder(node. left)
        postorder(node.right)
        visit(node)
```

Result: D, E, B, F, C, A

4. Level-order (BFS): Uses queue; visits level by level Result: A, B, C, D, E, F

BST Operations

Search O(log n average, O(n) worst):

```
Python
def search(node, key):
    if node is None or node.key == key:
        return node
    if key < node.key:
        return search(node.left, key)
```

```
    return search(node.right, key)
```

Insert O(log n):

```
Python
def insert(node, key):
    if node is None:
        return Node(key)
    if key < node.key:
        node.left = insert(node. left, key)
    else:
        node.right = insert(node.right, key)
    return node
```

Delete O(log n): Three cases:

1. Leaf node: Simply remove
2. One child: Replace with child
3. Two children: Replace with inorder successor

---

15. PART 1 Basic concepts of object-oriented paradigm. Class, object, instantiation. Inheritance, class hierarchy. Polymorphism, method overloading. Scoping, information hiding, accessibility levels. Abstract classes and interfaces. Class diagram of UML. PART 2 The architectures and algorithm elements that define the

operation of SNMP and RMON network management systems.

Part 1

Object-Oriented Programming (OOP)

OOP is a programming paradigm based on objects that contain data (attributes) and code (methods).

Four Pillars of OOP:

1. **Encapsulation** - Bundle data and methods, hide internals
2. **Abstraction** - Hide complexity, show only essential features
3. **Inheritance** - Create new classes from existing ones
4. **Polymorphism** - Same interface, different implementations

Class

A class is a blueprint or template for creating objects.

```
Java
public class Car {
    // Attributes (fields)
    private String color;
    private int speed;

    // Constructor
    public Car(String color) {
        this. color = color;
        this.speed = 0;
    }

    // Methods
    public void accelerate() {
        speed += 10;
```

```
    }

    public int getSpeed() {
        return speed;
    }
}
```

Object

An object is an instance of a class - a concrete entity with state and behavior.

```
Java
Car myCar = new Car("Red");  // Object creation
myCar. accelerate();          // Method call
```

Instantiation

Instantiation is the process of creating an object from a class.

Steps:

1. Memory allocation
2. Constructor execution
3. Reference assignment

```
Java
// Instantiation with 'new' keyword
Student s1 = new Student("John", 20);
Student s2 = new Student("Jane", 22);
```

Inheritance

Inheritance allows a class to inherit attributes and methods from another class.

```
Java
// Parent class
public class Animal {
    protected String name;
```

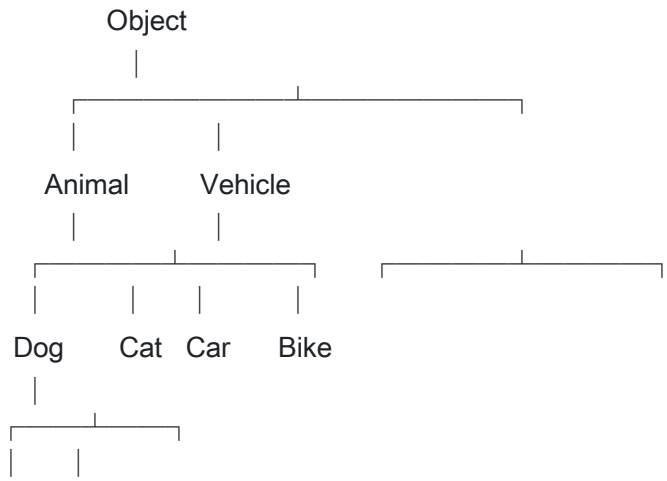
```
    public void eat() {
        System.out.println("Eating...");
    }
}

// Child class
public class Dog extends Animal {
    public void bark() {
        System.out.println("Woof!");
    }

    @Override
    public void eat() {
        System.out.println("Dog eating...");
    }
}
```

Class Hierarchy

Code



Types:

- Single inheritance: One parent class
- Multiple inheritance: Multiple parents (supported via interfaces in Java)
- Multilevel: Chain of inheritance
- Hierarchical: One parent, multiple children

Polymorphism

Polymorphism allows objects of different types to be treated through the same interface.

1. Compile-time (Static) - Method Overloading: Same method name, different parameters.

```
Java
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }

    public double add(double a, double b) {
        return a + b;
    }

    public int add(int a, int b, int c) {
        return a + b + c;
    }
}
```

2. Runtime (Dynamic) - Method Overriding: Child class redefines parent method.

```
Java
Animal animal = new Dog();  // Polymorphic reference
animal.eat();               // Calls Dog's eat() method
```

Scoping

Scope defines where a variable or method can be accessed.

Types:

- **Class scope** - Accessible throughout the class
- **Method scope** - Local to the method
- **Block scope** - Local to a block ({})

```
Java
public class Example {
```

```
    private int classVar;          // Class scope

    public void method() {
        int methodVar = 10;        // Method scope

        if (true) {
            int blockVar = 20;     // Block scope
        }
        // blockVar not accessible here
    }
}
```

Information Hiding

Information hiding protects internal state from unauthorized access.

Benefits:

- Prevents accidental modification
- Allows implementation changes
- Reduces dependencies

Accessibility Levels

Modifier	Class	Package	Subclass	World
private	✓	✗	✗	✗
default	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

Abstract Classes

Abstract classes cannot be instantiated and may contain abstract methods.

Java

```java
public abstract class Shape {
    protected String color;

    // Abstract method (no implementation)
    public abstract double getArea();

    // Concrete method
    public void setColor(String color) {
        this. color = color;
    }
}

public class Circle extends Shape {
    private double radius;

    @Override
    public double getArea() {
        return Math.PI * radius * radius;
    }
}
```

Interfaces

Interfaces define a contract of methods that implementing classes must provide.

Java

```java
public interface Drawable {
    void draw();          // Implicitly public abstract

    default void print() { // Default method (Java 8+)
        System.out.println("Printing...");
    }
}

public class Rectangle implements Drawable {
```

```
    @Override
    public void draw() {
        System.out.println("Drawing rectangle");
    }
}
```

Abstract Class vs Interface:

Feature	Abstract Class	Interface
Instantiation	No	No
Inheritance	Single	Multiple
Methods	Abstract + Concrete	Abstract (+ default)
Variables	Any type	public static final
Constructor	Yes	No

UML Class Diagrams

UML class diagrams visualize class structure and relationships.

Class Box:

```
Code
┌─────────────────────────────────────────┐
│      ClassName          │ ← Class name  │
├─────────────────────────────────────────┤
│ - privateAttr:  int     │ ← Attributes  │
│ + publicAttr: String    │               │
├─────────────────────────────────────────┤
│ + method(): void        │ ← Methods     │
│ - helper(): int         │               │
└─────────────────────────────────────────┘
```

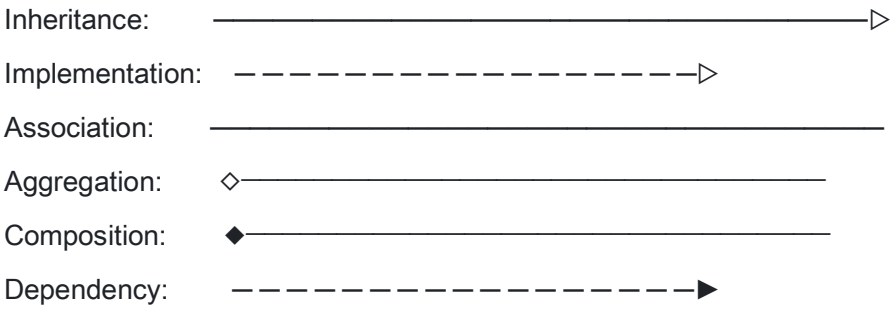
Visibility Symbols:

- + Public
- - Private

- # Protected
- ~ Package

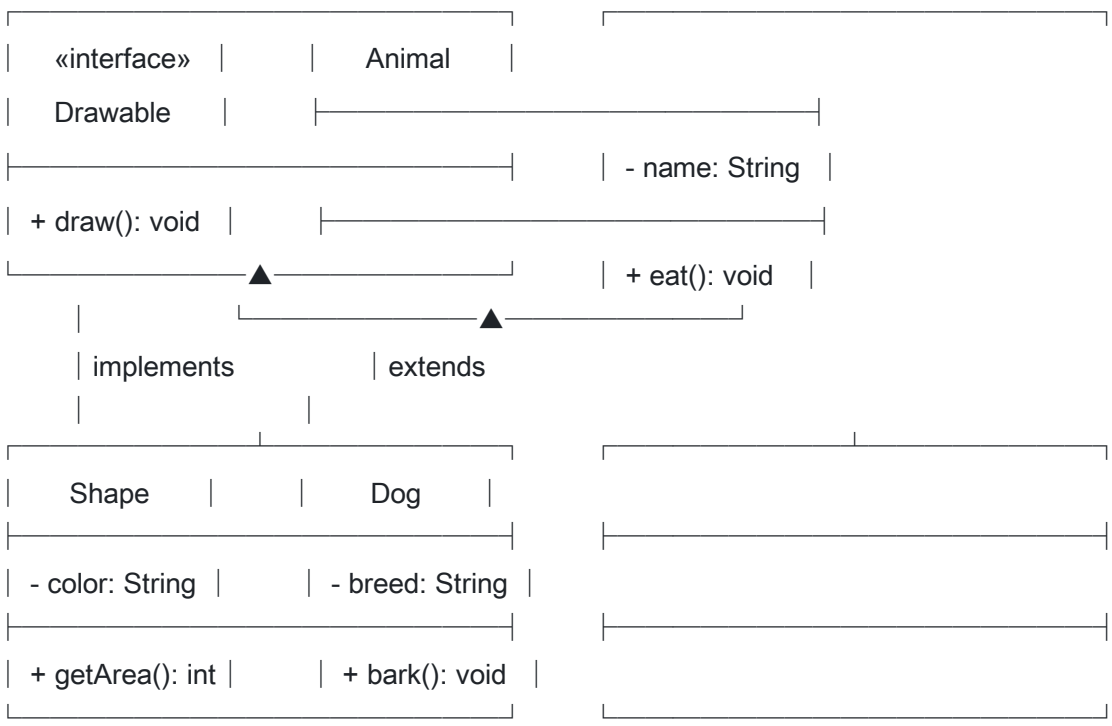
Relationships:

Code



Example UML Diagram:

Code



Part 2

SNMP (Simple Network Management Protocol)

SNMP is a protocol for monitoring and managing network devices.

Components:

1. **Managed Devices:**
    - Routers, switches, servers
    - Run SNMP agent software
2. **SNMP Agent:**
    - Software on managed device
    - Collects and stores management information
    - Responds to manager requests
3. **SNMP Manager:**
    - Centralized management station
    - Queries agents
    - Receives notifications
4. **Management Information Base (MIB):**
    - Hierarchical database of objects
    - Object Identifier (OID) for each variable

MIB Tree Structure:

Code

```
iso(1)
  └── org(3)
      └── dod(6)
          └── internet(1)
              ├── mgmt(2)
              │   └── mib-2(1)
              │       ├── system(1)
              │       ├── interfaces(2)
              │       └── ...
              └── private(4)
                  └── enterprises(1)
```

SNMP Operations:

Operation	Description
GET	Retrieve single object value
GET-NEXT	Get next object in MIB
GET-BULK	Get multiple objects (v2+)
SET	Modify object value
TRAP	Unsolicited notification
INFORM	Acknowledged notification (v2+)

SNMP Message Format:

```
Code
┌──────────────────────────────────────────────────────────────┐
│ Version │ Community │ PDU Type │ Data  │
└──────────────────────────────────────────────────────────────┘
```

SNMP Versions:

- **SNMPv1:** Basic, community string authentication
- **SNMPv2c:** Improved performance, bulk operations
- **SNMPv3:** Security (authentication, encryption, access control)

RMON (Remote Monitoring)

RMON extends SNMP for network traffic monitoring.

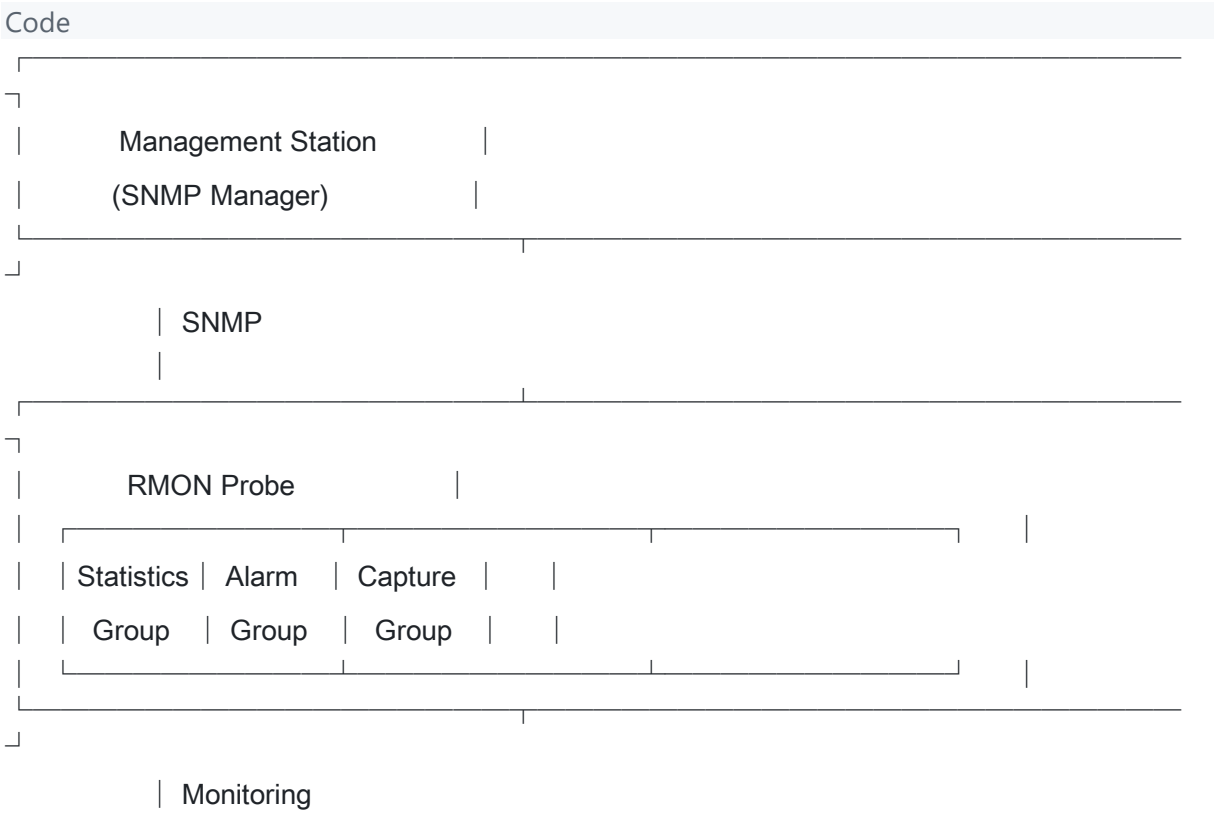
Purpose:

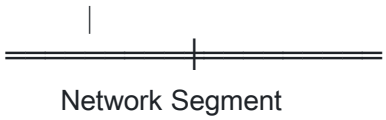
- Monitor network segments remotely
- Reduce management traffic
- Provide historical data
- Alarm and event management

RMON Groups (RFC 1757):

Group	Description
Statistics	Traffic statistics per interface
History	Historical samples
Alarm	Threshold monitoring
Host	Per-host traffic statistics
HostTopN	Top N hosts by statistic
Matrix	Host pair communication
Filter	Packet filtering
Capture	Packet capture
Event	Event generation

RMON Architecture:





SNMP vs RMON

Feature	SNMP	RMON
Focus	Device-oriented	Network-oriented
Data	Current status	Historical trends
Analysis	Manager does analysis	Probe does analysis
Traffic	Higher (polls each device)	Lower (probe aggregates)
Scope	Individual devices	Network segments
Proactive	Reactive (polling)	Proactive (alarms)

RMON2:

- Extends to higher layers (Network, Application)
- Protocol distribution analysis
- Application-level monitoring

Summary

This comprehensive guide covers all 15 examination topics from the Computer Science Engineering BSc (2021) Final Exam. Each question is structured with:

- **Clear headings and sections** for easy navigation
- **Detailed explanations** of concepts
- **Diagrams and figures** where applicable
- **Code examples** in relevant programming languages
- **Comparison tables** for related concepts
- **Practical applications** of theoretical concepts